

Scaling Diffusion Models to Large Sparse Graphs

A Major Qualifying Project (MQP) Report
Submitted to the Faculty of
WORCESTER POLYTECHNIC INSTITUTE
in partial fulfillment of the requirements
for the Degree of Bachelor of Science in

Computer Science,
Data Science,
Mathematical Sciences

By:

David Dechantsreiter
Weaver Goldman
Botao Hu
Hien Pham

Project Advisors:

Fabricio Murai
Oren Mangoubi

Date: March 6 2026

This report represents work of WPI undergraduate students submitted to the faculty as evidence of a degree requirement. WPI routinely publishes these reports on its website without editorial or peer review. For more information about the projects program at WPI, see <http://www.wpi.edu/Academics/Projects>.

Abstract

Generative modeling for graph-structured data is critical for advancing fields ranging from drug discovery to social network analysis. However, scaling these models to large graphs remains a significant challenge. Existing graph diffusion models treat graphs as dense adjacency matrices, explicitly modeling all possible edges. Consequently, this leads to prohibitive quadratic computational complexity, restricting most current models to small graphs with only a few hundred nodes. This project addresses this scalability limitation through two distinct contributions. First, we introduce SCALE-MGD, which applies masked discrete diffusion directly to a sparse edge-list representation using a GNN-based architecture, achieving linear memory complexity ($O(n+m)$), where n is the number of nodes and m is the number of edges. Second, we introduce SparserDiff, a memory efficient adaptation of an existing method (SparseDiff) that reformulates the original forward noising process to avoid materializing the full set of non-existent edges. SCALE-MGD exhibits better scalability, than all prior models, however struggles with generation fidelity. SparserDiff generates high-fidelity graphs, much like its predecessor SparseDiff; however, it does not scale as well as SCALE-MGD. Finally, we introduce RedditWalk, a new large-scale benchmark derived from the Reddit Hyperlink Network, enabling variable-size graph generation that preserves structural characteristics across scales. Furthermore, we evaluate SparserDiff on three sampled datasets from RedditWalk, demonstrating the potential of the RedditWalk dataset as a benchmark for large-scale graph generation.

Acknowledgements

Our team would like to thank Professor Murai and Professor Mangoubi for providing our team with all the feedback and support for our project this past term. We would also like to thank Sajal Chakroborty for assisting initial team discussions and providing mathematical explanations. Finally, we would like to thank the Academic & Research Computing group at Worcester Polytechnic Institute for providing the computational resources required for this project.

Contents

1	Introduction	1
2	Background	3
2.1	Graph Machine Learning	4
2.1.1	The Geometric Nature of Graphs	4
2.1.2	Permutation Symmetry and Inductive Biases	4
2.2	Evolution of Graph Encoders: From Local to Global	5
2.2.1	Local Interaction: Message Passing Neural Networks (MPNNs)	5
2.2.2	Global Interaction: Graph Transformers	6
2.2.3	Generative Models on Graphs: Challenges and Frontiers	6
2.2.3.1	The Mismatch of Continuous Gaussian Diffusion	7
2.3	Generative Models Overview	7
2.3.1	Variational Autoencoders (VAEs)	7
2.3.2	Generative Adversarial Networks (GANs)	8
2.3.3	Autoregressive Models	8
2.3.4	Why Diffusion Models?	9
2.4	Diffusion Models	9
2.4.1	Forward Process: From Data to Noise	9
2.4.2	Reverse Process: Learning to Denoise	10
2.5	Limitations of Continuous Diffusion for Discrete Data	11
2.6	Discrete Diffusion	12
2.6.1	DiGress	12
2.6.2	Masked Diffusion	15
3	Related Work	17
3.1	MG-Diff	17
3.2	SparseDiff	18
4	Methods	19
4.1	SCALE-MGD	20
4.1.1	Sparse Graph Representation	20
4.1.2	Noise Model	21
4.1.3	Loss Calculation	22
4.1.4	Neural Network Model	23
4.1.5	Training	27
4.1.6	Sampling	27
4.2	SparsifierDiff: A Memory-Efficient Variant of SparseDiff	28
4.2.1	Space Complexity Analysis	28
4.2.2	Time Complexity Analysis	29

5	Experiments and Results	31
5.1	Evaluating Memory and Time Scaling	31
5.1.1	Experiment Settings	31
5.1.2	Results and Discussion	33
5.2	SparserDiff Evaluation	42
5.2.1	Experimental Design	42
5.2.2	Results and Discussion	44
5.3	SCALE-MGD Evaluation	49
5.3.1	Experimental Design	49
5.3.2	Results and Discussion	49
6	RedditWalk: A Large-Scale Benchmark Dataset	54
6.1	Graph Structure and Giant Component Filtering	54
6.2	Node Feature Extraction via Textual Descriptions	55
6.3	Subgraph Sampling for Dataset Construction	57
6.4	Evaluation of SparserDiff on Subsampled RedditWalk	59
7	Future Work	62
8	Conclusion	63
	References	64

List of Tables

1	Hyperparameters of the neural networks used in the complexity analysis.	31
2	Graph type specifications used in the experiments (batch size = 128 for all).	32
3	Molecular Graph ($N = 20$)	36
4	Small Social Network ($N = 50$)	37
5	Medium Sized Graph ($N = 100$)	38
6	Large Graph ($N = 200$)	39
7	Complex Edge Types ($N = 50, d_E = 8$)	40
8	Complex Node Types ($N = 50, d_X = 20$)	41
9	Experiment design parameters for sampling-scaling benchmark.	42
10	Experiment 2: Edge sampling benchmark on the Ego test set (151 graphs, 50–364 nodes). Median wall-clock time per batch; memory is the peak theoretical allocation.	46
11	Experiment 2: Edge sampling benchmark on the QM9 test set (13,054 molecules, 3–29 nodes). On small graphs, the original SparseDiff vectorized approach is faster; SparserDiff still reduces memory 4–7×.	46
12	Experiment 3: Molecular generation quality on QM9 (20 epochs, no H). Bold indicates the better result. $\uparrow$ = higher is better; $\downarrow$ = lower is better.	47
13	Experiment 3: Graph generation quality on Ego (100 epochs). All MMD metrics: lower is better ($\downarrow$). Bold indicates the better result.	48
14	Molecular generation performance. Higher is better for all reported metrics.	52

15	Sample quality on Ego and Planar datasets (unscaled MMD values). Lower is better for all metrics.	53
16	Mean pairwise Jaccard similarity across sampled subgraphs.	59
17	SparsifierDiff Spectre metrics on datasets sampled from RedditWalk at three scales, after 1000 training epochs.	60

List of Figures

1	The Diffusion Framework is defined by an information-destructing forward process and a reconstructing reverse generative process. The reverse generative process is performed by a neural network, trained to estimate the clean signal x_0 . The model’s estimate $\hat{x}_0$ is then plugged into the posterior distribution $q(x_{t-1} x_t, x_0)$ from which x_{t-1} is sampled.	3
2	DiGress Diffusion as visualized in [Vignac et al., 2022]. The forward process is shown on top. Transitions from the clean graph G_0 to any timestep t are obtained using the cumulative transition matrix $\bar{Q}_t$. The reverse process is shown below. The true reverse distribution $q(G_{t-1} G_0, G_t)$, depends on the unknown clean graph G_0 . A neural network is therefore trained to predict a surrogate distribution $p_\theta(G G_t)$ over clean graphs. This prediction is combined with the known diffusion posterior to form $p_\theta(G_{t-1} G_t)$ from which a graph at timestep $t - 1$ is sampled. The network is trained using a cross-entropy loss between the predicted clean graph distribution and the true clean graph.	13
3	Before the noisy graph is fed to the GNN encoder, the masked edges are randomly populated according to the dataset marginal distributions for each edge tuple component.	24
4	Nonexistent edges in the adjacency matrix (A) are sampled by computing a prefix array of the number of missing edges before each node and sampling indices of nonexistent edges (B) and utilizing binary search to identify the real edge corresponding to each sampled index (C), leading to a set of new sampled edges (D).	29
5	VRAM usage comparison across different algorithms and stages with respect to the number of nodes on synthetic graphs. Trends indicate the space complexity of each method.	33
6	Memory usage and sampling time comparison between different new-edge-sampling algorithms.	45
7	Experiment 2: Sampling speed (left) and peak memory usage (right) on the Ego dataset. SparsifierDiff avoids the quadratic bottleneck of the original method, maintaining near-constant memory overhead while achieving significant speedups.	46
8	Distributions of edge counts from generated graphs and from the original dataset.	50
9	Distributions of triangle counts from generated graphs and from the original dataset.	51
10	PCA visualization of the BERT embeddings for 46,668 subreddits. The high-dimensional semantic vectors are projected onto the first two principal components, colored by their assigned cluster ID (0 to 19).	56
11	t-SNE visualization of Node2Vec structural embeddings for 46,668 subreddits in the GWCC. Nodes are color-coded by their corresponding BERT-derived semantic cluster IDs. The visual grouping of colors demonstrates a strong homophily effect, where topological communities align closely with semantic topics.	57
12	The sampled subgraphs form a coherent structural data distribution	58
13	Degree distribution (left) and clustering coefficient distribution (right) for test graphs versus generated graphs across three graph scales. Each faint line represents a single graph; the bold line is the aggregate distribution.	61

14	Spring layout visualization of a sampled graph at t=480, t=490, and t=500. At t=480 several nodes remain connected to the main component. By t=490 a node has already detached; in the final 10 steps, all remaining bridging edges are removed, leaving multiple nodes fully isolated.	61
----	---	----

1 Introduction

Graphs are a fundamental abstraction used for representing complex systems composed of objects (or entities) and their relationships, ranging from molecular interactions in chemistry to vast social networks and citation graphs [Niu et al., 2020]. Due to their expressive power, there is a growing interest in *generative models* capable of synthesizing realistic graph structures. However, a critical gap exists in the current literature: while models for small, dense graphs (like molecules) have matured, generating large, sparse graphs remains an open challenge. Most existing approaches struggle to scale, facing prohibitive computational costs when processing graphs with thousands of nodes, limiting their applicability to real-world data.

This scalability bottleneck presents a significant hurdle for scientific and industrial applications. In domains such as social network analysis or knowledge graph completion, graphs naturally exhibit sparse connectivity but massive scale. Developing generative models that can efficiently learn distributions over such large sparse structures is crucial for tasks like synthetic data generation for privacy preservation, anomaly detection in large networks, and unsupervised representation learning [Huang et al., 2025, Nguyen and Fang, 2024]. Without scalable generative methods, these powerful deep learning techniques remain inaccessible for macro-scale graph data.

One of the most recent approaches to generating graph structures is the family of graph diffusion models, which learn to generate data by gradually corrupting it and learning to reverse that process. These models are based upon the concept of the Denoising Diffusion Probabilistic Models (DDPM), introduced by [Ho et al., 2020], which defines two processes: (1) a **forward noising process** that gradually corrupts the data by adding noise over a sequence of time steps; (2) a **reverse process** that learns to iteratively denoise it, transforming samples from a simple prior (e.g., standard Gaussian or Uniform) into realistic outputs [Ho et al., 2020, Sohl-Dickstein et al., 2015].

While this diffusion framework has achieved remarkable success in continuous domains such as image generation, it was not originally designed for discrete data and, in particular, for graphs. Graphs pose unique challenges, such as variable size, permutation invariance, and discrete node/edge features, that violate the assumptions of standard continuous diffusion processes [Wang et al., 2025]. In particular, the discrete and often sparse nature of real-world graphs is fundamentally misaligned with the continuous, dense, and Gaussian noise distributions used in standard diffusion models. As a result, many graph diffusion approaches that apply continuous Gaussian noise to graph embeddings or adjacency representations tend to generate overly connected networks that fail to capture the structural characteristics of real graphs [Vignac et al.,

2022, Huang et al., 2023].

These challenges have motivated discrete diffusion methods for graphs, including DiGress [Vignac et al., 2022] and mask-based variants such as MGDiff [Zhang et al., 2025]. A remaining bottleneck is scalability: many discrete approaches operate over the full adjacency matrix and explicitly represent missing connections (e.g., as a “non-edge” type), resulting in $O(n^2)$ memory and computation in the number of nodes, much of which is spent modeling absent edges in sparse graphs [Qin et al., 2025]. Another proposed approach, SparseDiff, instead operates on a sparse edge/node list representation, avoiding an explicit $n \times n$ adjacency matrix and thereby reducing the need to represent absent edges as a separate edge type, a design-choice that in prior approaches effectively forces the model to operate over a fully connected graph regardless of the input’s true sparsity Qin et al. [2025]. However, due to its noise model, SparseDiff still had to materialize the set of non-existent edges, resulting in significant memory constraints.

In this work, we propose two diffusion models: SCALE-MGD (Scalable Masked Graph Diffusion), which performs masking directly on a graph’s sparse representation, and SparserDiff, a more efficient variant of SparseDiff with an improved forward noising process. We provide an empirical analysis of the scaling and modeling limitations faced by current methods and show that SCALE-MGD and SparserDiff take concrete steps toward mitigating these issues. We further evaluate both approaches in a case study using a new dataset derived from the Reddit Hyperlink Network. Most notably, SparserDiff corrects key deficiencies in SparseDiff’s forward process by avoiding materialization of all possible edges while preserving the state-of-the-art performance originally reported for SparseDiff.

Goal

The primary goal of this project is to develop a scalable generative framework capable of synthesizing large-scale sparse graphs. Specifically, we aim to address the limitations of continuous diffusion models, which typically destroy graph sparsity during the noising process, leading to prohibitive $O(n^2)$ computational costs. By introducing a masked discrete diffusion mechanism, our approach seeks to preserve the sparse structure throughout the generation pipeline, thereby enabling efficient training and sampling on graphs with thousands of nodes while maintaining high structural fidelity.

Objectives

Our project aims to achieve the goal of scaling diffusion models to large sparse graphs through the following four objectives:

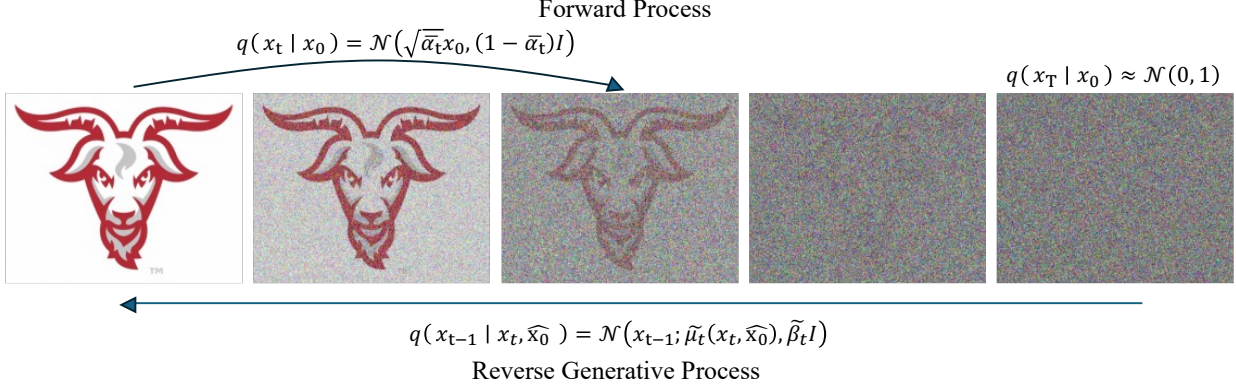


Figure 1: The Diffusion Framework is defined by an information-destructing forward process and a re-constructing reverse generative process. The reverse generative process is performed by a neural network, trained to estimate the clean signal x_0 . The model’s estimate $\hat{x}_0$ is then plugged into the posterior distribution $q(x_{t-1}|x_t, x_0)$ from which x_{t-1} is sampled.

1. **Analyze Limitations:** Investigate the computational bottlenecks of continuous Gaussian diffusion models when applied to discrete graph data, specifically regarding the loss of sparsity.
2. **Implement Discrete Diffusion:** Develop a Masked Discrete Diffusion framework that operates directly on the discrete state space of graphs (nodes and edges).
3. **Optimize for Sparsity:** Integrate sparse matrix operations into the diffusion pipeline to ensure the model complexity scales linearly with the graph size ($O(n + m)$) rather than quadratically ($O(n^2)$).
4. **Evaluate Performance:** Compare the generated graphs against real-world benchmarks to assess both structural fidelity and computational efficiency (training time and memory usage) on large-scale datasets.

2 Background

Diffusion models are a class of generative models that learn complex data distributions by reversing a gradual noising process. They are formulated in terms of two complementary components: a forward (diffusion) process and a reverse (generative) process Prince [2023], as illustrated in Figure 1. In the forward process, data samples are progressively corrupted with noise, typically until they approximate a simple prior distribution. The reverse process is parameterized by a neural network trained to approximate the inverse of this noising procedure and to iteratively transform samples from the prior into novel samples that resemble the training data.

Initially, diffusion generative models were designed to generate realistic high-dimensional samples in Euclidean spaces, such as images [Sohl-Dickstein et al., 2015]. However, since then the general framework has been extended to generate graphs, text, and molecules (see [Bilodeau et al., 2022, Gong et al., 2023, Vignac et al., 2022]). In this section, we first review continuous diffusion models, then discuss the limitations of applying them directly to discrete structured data such as graphs, and finally introduce discrete diffusion frameworks that operate natively in discrete spaces.

2.1 Graph Machine Learning

To understand the challenges of generating large-scale graphs, we must first establish the mathematical properties that distinguish graphs from other data modalities like images or text. Unlike grid-based data which benefits from strong spatial regularity, graphs are irregular structures that require specialized geometric priors. This section systematically explores the nature of graph data, the evolution of neural architectures designed to process them, and the specific limitations of current generative paradigms that necessitate a new approach.

2.1.1 The Geometric Nature of Graphs

Standard deep learning has achieved remarkable success on Euclidean data: images reside on fixed 2D grids, and text forms 1D sequences. In these domains, geometric concepts like “shift” or “neighbor” are globally consistent (e.g., the pixel to the right always exists).

However, graphs exist in a **Non-Euclidean** domain. A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is defined strictly by the topological connections between its nodes $\mathcal{V}$ and edges $\mathcal{E}$, lacking any inherent spatial coordinate system. This irregularity imposes a fundamental constraint on any learning algorithm: the model must be robust to the arbitrary ordering of nodes.

2.1.2 Permutation Symmetry and Inductive Biases

The most critical challenge in Graph ML is **Permutation Symmetry**. Consider a social network of three people: Alice, Bob, and Charlie. Whether we represent this data in a computer’s memory as $[Alice, Bob, Charlie]$ or $[Charlie, Alice, Bob]$ is irrelevant to the underlying social structure. However, standard neural networks (like MLPs or RNNs) map specific inputs to specific weights, making them sensitive to this arbitrary ordering.

To operate effectively on graphs, neural networks must possess specific geometric inductive biases:

Definition 1 (Permutation Invariance and Equivariance). *Let $\mathbf{A} \in \{0, 1\}^{N \times N}$ be the adjacency matrix and $\mathbf{X} \in \mathbb{R}^{N \times F}$ be the node feature matrix. Let $\mathbf{P}$ be any arbitrary permutation matrix.*

- **Permutation Invariance** (Required for graph-level tasks): *The output remains unchanged regardless of node ordering:*

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = f(\mathbf{A}, \mathbf{X}) \quad (1)$$

Example: Classifying a molecule as toxic or non-toxic should not depend on the order in which its atoms are indexed.

- **Permutation Equivariance** (Required for node-level tasks): *If the input nodes are permuted, the output features are permuted in the exact same way:*

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = \mathbf{P}f(\mathbf{A}, \mathbf{X}) \quad (2)$$

Example: If “User A” moves to the 5th row in the input matrix, their predicted credit score must also move to the 5th row in the output.

2.2 Evolution of Graph Encoders: From Local to Global

Designing functions that satisfy the equivariance property while remaining expressive has been the central theme of Graph Representation Learning. Once permutation equivariance is achieved at the node level, graph-level permutation invariance can subsequently be obtained for free. Specifically, if the intermediate node representations are permutation equivariant, applying a permutation-invariant aggregation function, such as a sum or mean, will naturally result in a permutation-invariant representation for the entire graph.

2.2.1 Local Interaction: Message Passing Neural Networks (MPNNs)

The dominant paradigm for learning on graphs is Message Passing [Gilmer et al., 2017]. Inspired by the Weisfeiler-Lehman graph isomorphism test, MPNNs update node representations by iteratively aggregating information from local neighborhoods. For a node u , the update rule at layer k is given by:

$$\mathbf{h}_u^{(k)} = \phi^{(k)} \left(\mathbf{h}_u^{(k-1)}, \bigoplus_{v \in \mathcal{N}(u)} \psi^{(k)}(\mathbf{h}_u^{(k-1)}, \mathbf{h}_v^{(k-1)}, \mathbf{e}_{uv}) \right) \quad (3)$$

where $\mathcal{N}(u)$ is the set of neighbors of node u , $\oplus$ is a permutation-invariant aggregation function (e.g., Sum, Mean, Max), and ϕ, ψ are learnable neural networks.

The Bottleneck: While MPNNs are computationally efficient with linear complexity $O(m)$, they suffer from **Oversquashing**. To pass information between distant nodes, the signal must propagate through intermediate neighbors. As the receptive field grows exponentially, information from the entire graph must be compressed into a fixed-size vector, causing a “bottleneck” where long-range dependencies are lost [Alon and Yahav, 2021].

2.2.2 Global Interaction: Graph Transformers

While MPNNs are computationally efficient, the oversquashing bottleneck necessitates a shift towards global attention architectures. To resolve the long-range dependency problem, recent research has pivoted towards Graph Transformers [Dwivedi and Bresson, 2020, Ying et al., 2021]. Unlike MPNNs, these Transformer-based Graph Neural Networks allow every node to attend to every other node directly, effectively treating the graph as a fully connected structure with “soft” attention weights. The attention weight between nodes u and v represented by vectors $\mathbf{h}_u$ and $\mathbf{h}_v$ is

$$\text{Attention}(u, v) \propto \exp \left(\frac{(\mathbf{W}_Q \mathbf{h}_u)^\top (\mathbf{W}_K \mathbf{h}_v)}{\sqrt{d}} \right), \quad (4)$$

where $\mathbf{W}_Q$ and $\mathbf{W}_K$ are learnable weight matrices that project the input node representations into query and key spaces respectively, and d is the dimensionality of these projected vectors, which acts as a scaling factor to stabilize gradients during training. Since standard attention is purely set-based (ignoring graph structure), Graph Transformers must inject structural information via *Positional Encodings* (e.g., Laplacian eigenvectors) to distinguish nodes.

The Scalability Wall: While expressive, the full attention mechanism requires computing an $N \times N$ attention matrix, leading to quadratic complexity $O(n^2)$. This is prohibitive for large-scale graphs ($N > 10,000$), forcing a trade-off between capturing global structure and computational feasibility.

2.2.3 Generative Models on Graphs: Challenges and Frontiers

Generative modeling extends representation learning by attempting to learn the complex joint distribution $p(\mathcal{G})$ to synthesize new, realistic graphs. This task is significantly harder than classification tasks on graphs due to the discrete and combinatorial nature of graph topology.

2.2.3.1 The Mismatch of Continuous Gaussian Diffusion

Early attempts applied standard Denoising Diffusion Probabilistic Models (DDPM) directly to the adjacency matrix $A \in \{0, 1\}^{N \times N}$. The forward process adds Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to these discrete binary entries:

$$A_t = \sqrt{\bar{\alpha}_t} A_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (5)$$

This approach suffers from a fundamental “Manifold Mismatch”. Graphs are discrete objects, but Gaussian diffusion forces them into a continuous space.

1. **Discretization Error:** The reverse process predicts continuous values which must be rounded back to discrete edges, often leading to valid but incoherent structures.
2. **Destruction of Sparsity:** A real-world adjacency matrix is extremely sparse ($|\mathcal{E}| \ll N^2$). However, adding Gaussian noise instantly destroys this sparsity, turning A_t into a dense matrix where almost every entry is non-zero. This forces the neural network to process $O(N^2)$ interactions, making it impossible to scale to large graphs.

2.3 Generative Models Overview

Generative modeling on graphs aims to learn the underlying distribution $p_{\text{data}}(\mathcal{G})$ from a set of observed graphs and sample new structures $\mathcal{G}_{\text{new}} \sim p_{\text{model}}(\mathcal{G})$ that preserve the statistical and topological properties of the training set. Before detailing diffusion models, we briefly review three major families of deep generative models and their limitations when applied to graph data.

2.3.1 Variational Autoencoders (VAEs)

Variational Autoencoders [Kingma and Welling, 2013] learn a low-dimensional latent representation of graphs by maximizing the Evidence Lower Bound (ELBO). Approaches like GraphVAE [Simonovsky and Komodakis, 2018] encode the graph into a latent vector $\mathbf{z}$ and use a decoder to reconstruct the adjacency matrix.

Limitations: The primary challenge for Graph VAEs is the *reconstruction loss*. Since graphs are permutation invariant, calculating the difference between the input graph G and the reconstructed graph $\hat{G}$ requires solving the Graph Matching problem, which is NP-hard. Approximations exist but are

computationally expensive ($O(n^3)$ or higher), limiting VAEs to very small graphs.

2.3.2 Generative Adversarial Networks (GANs)

GANs [Goodfellow et al., 2014] employ a min-max game between a generator creating fake data points and a discriminator trying to distinguish them from real ones. Models like MolGAN [De Cao and Kipf, 2018] and NetGAN [Bojchevski et al., 2018] adapted this framework to graph data.

Limitations: GANs are notoriously unstable to train; they tend to suffer from mode collapse [Goodfellow et al., 2014, Salimans et al., 2016]. Furthermore, applying GANs to discrete graph data poses a fundamental difficulty: the discrete nature of edges (0 or 1) prevents gradients from backpropagating from the discriminator to the generator [Yu et al., 2017]. This necessitates techniques like Reinforcement Learning (e.g., REINFORCE) or continuous relaxation, which often degrade sample quality.

2.3.3 Autoregressive Models

Autoregressive models decompose the joint probability of the graph into a sequence of conditional probabilities. For example, GraphRNN [You et al., 2018] generates a graph node-by-node and edge-by-edge, treating the adjacency matrix as a sequence:

$$p(G) = \prod_{i=1}^N p(v_i \mid v_{<i}) \quad (6)$$

where v_i represents the i -th generated node along with its connectivity to the previous nodes, and $v_{<i}$ denotes the subgraph composed of all previously generated nodes $\{v_1, v_2, \dots, v_{i-1}\}$. In this context, the conditional probability distribution $p(v_i \mid v_{<i})$ defines the likelihood of adding node v_i and forming its specific edge connections to the existing graph structure, conditioned entirely on the exact topology of the subgraph generated up to step $i - 1$.

Limitations: These models inherently assume an ordering of nodes (linearization). However, graphs usually do not have a canonical order. To handle this, the model must learn to be robust to all possible permutations ($N!$ combinations), which is inefficient. Moreover, the strictly sequential generation process precludes parallelization, creating a severe computational bottleneck. Unlike diffusion models, which can generate components in parallel, autoregressive models are forced to generate node-by-node, making them unable to scale to large graphs with thousands of nodes.

2.3.4 Why Diffusion Models?

As discussed, previous approaches struggle with the unique properties of graphs: VAEs face expensive matching costs, GANs suffer from discrete gradient issues, and Autoregressive models are limited by sequential ordering.

Diffusion models offer a compelling alternative. By defining the generation process as an iterative denoising procedure, they avoid the need for explicit graph matching (unlike VAEs) and do not require adversarial training (unlike GANs). Moreover, as we will explore in the following sections, recent advances in discrete diffusion allow for preserving the permutation symmetry and sparsity of graphs, making them a suitable candidate for large-scale graph generation.

2.4 Diffusion Models

To formalize the ideas underlying diffusion, we begin with the Denoising Diffusion Probabilistic Model (DDPM) proposed by [Ho et al., 2020]. This framework provides a clear mathematical description of how diffusion models operate in continuous data domains. The core idea is to define a forward process that gradually destroys data structure, and then to learn a reverse process to restore it. The network parameters are optimized by minimizing a denoising loss function, which acts as a simplified objective derived from the variational lower bound on the data log-likelihood [Sohl-Dickstein et al., 2015].

2.4.1 Forward Process: From Data to Noise

Because early diffusion models were developed for continuous Euclidean domains such as images, their forward processes are most often defined using Gaussian noise applied to real-valued data. The goal of the forward 'diffusion' process is to collapse the initial complex data distribution $q(x_0)$ into a simple known prior distribution, often a standard Gaussian or Uniform distribution [Sohl-Dickstein et al., 2015]. This transformation is achieved through a fixed corruption process defined as a Markov chain, which generates a sequence of increasingly noisy samples $x_1, \dots, x_T$ by sequentially adding noise to the given sample x_0 at each time step $t \in \{1, 2, \dots, T\}$ [Ho et al., 2020]. The joint density of the noisy samples conditioned on the original sample is given by:

$$q(x_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1}), \quad (7)$$

The amount of noise added at each step is according to a noise schedule $\beta_t \in (0, 1)$. The posterior distribution $q(x_t|x_{t-1})$ is given by

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I), \quad \alpha_t = 1 - \beta_t. \quad (8)$$

A crucial property of diffusion models is that the marginal transition distribution $q(x_t|x_0)$ has a closed-form solution, allowing direct sampling of x_t without explicitly simulating the full Markov chain [Vignac et al., 2022]. In continuous state spaces, one can derive a closed form for $q(x_t|x_0)$, which enables jumping to any time step as

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I), \quad (9)$$

where $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. Sampling from this distribution provides a data point that is corrupted by a certain amount of Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, I)$, given by

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon. \quad (10)$$

As the time step t increases, $\bar{\alpha}_t$ typically approaches 0 so that $q(x_T) \approx \mathcal{N}(0, I)$.

2.4.2 Reverse Process: Learning to Denoise

While the forward process gradually destroys structure in the data, the generative model learns to reverse this corruption. At inference time, a sample is drawn from the simple prior $\mathcal{N}(0, 1)$ and then progressively denoised back to the data distribution. This reverse trajectory is again formulated as a Markov chain, but unlike the forward process, its transitions must be learned [Ho et al., 2020].

In theory, one could recover the reverse conditional using Bayes' rule

$$q(x_{t-1}|x_t) = \frac{q(x_t|x_{t-1})q(x_{t-1})}{q(x_t)}. \quad (11)$$

In practice, however, the exact marginals $q(x_{t-1})$ and $q(x_t)$ are intractable because they require integrating over the unknown data distribution. Instead, the reverse process is trained to approximate the true reverse posterior $q(x_{t-1}|x_t, x_0)$, that is the distribution over less corrupted data (x_{t-1}) given a more-corrupted state (x_t) and the original data (x_0) [Sohl-Dickstein et al., 2015]. This distribution is analytically computable when conditioned on x_0 [Ho et al., 2020], a quantity available to the training objective. In the case of

continuous diffusion this makes it possible to express the reverse transition as a Gaussian whose mean depends on the model’s estimate of the underlying clean signal:

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}\left(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I\right), \quad (12)$$

where $\tilde{\beta}$ and $\tilde{\mu}$ are the posterior variance and the posterior mean respectively.

In practice, training draws a random timestep t , samples a corrupted state $x_t \sim q(x_t|x_0)$ and optimizes a neural network D_θ to predict the clean signal $\hat{x}_0$ [Ho et al., 2020, Vignac et al., 2022], as seen in Figure 1. Substituting this estimate into the analytic form of the reverse Gaussian yields a tractable transition distribution $p_\theta(x_{t-1} | x_t)$, which is used during sampling to progressively remove noise. Over many steps, this learned denoising chain transforms the initial Gaussian noise into a coherent data sample.

A particularly useful reformulation comes from writing x_t in terms of the original data and the injected noise because in Eq. 10 predicting x_0 is equivalent to predicting the noise ϵ that produced x_t . This leads to a remarkably simple and stable training objective: a mean-squared error between the true noise and the network’s prediction:

$$L(\theta) = \mathbb{E}_{x_0, \epsilon, t} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t) \right\|^2 \right]. \quad (13)$$

This “noise-prediction” formulation is one of the central reasons continuous diffusion models are easy to train and capable of generating high-quality samples despite their relatively simple architecture [Ho et al., 2020]. It also clearly separates the roles of the forward and reverse processes: the forward process creates a smooth continuum of noise levels, and the reverse model learns how to retrace that continuum back to clean data.

2.5 Limitations of Continuous Diffusion for Discrete Data

Diffusion models are naturally defined in continuous Euclidean spaces, where Gaussian noise provides a mathematically convenient corruption process. As a result, their direct application to discrete structures introduces several challenges [Schiff et al., 2025].

Loss of Structural Integrity Under Gaussian Corruption. To apply continuous diffusion to discrete data, models typically employ a process called dequantization, which embeds discrete graphs into a continuous state space by adding real-valued Gaussian noise [Kong et al., 2023]. This mechanism fundamentally distorts the distribution of the original discrete structures [Yin et al., 2025]. Rather than

preserving the sparse topology typical of real-world graphs, this corruption produces dense, fully connected matrices with weighted edges that no longer correspond to valid objects [Vignac et al., 2022]. Consequently, structural information, such as connectivity or cycle counts, is not defined for these intermediate states [Vignac et al., 2022]. Because these noisy states deviate substantially from the manifold of valid graphs, the reverse process must reconstruct complex structure from highly uninformative, distributionally distorted inputs making model training difficult [Kong et al., 2023].

Mismatch Between Continuous Outputs and Discrete Constraints. While dequantization allows for continuous noise to be used, the raw output consists of dense matrices of floating-point numbers that do not inherently represent a valid discrete object. To recover a usable graph, the model must apply a heuristic post-processing step to “snap” these continuous values back to discrete categories. For node features, this typically involves taking the argmax across feature dimensions [Vignac et al., 2022]. This projection introduces a fundamental disconnect between the training objective and the generation quality. A generated value may sit on a decision boundary, where a microscopic difference in continuous space results in a macroscopic change in graph topology. This can lead to a node-edge mismatch, where the projected edge structure is incompatible with the projected node types (e.g., an atom exceeding its valency limits) because the continuous manifold did not strictly enforce the logical dependencies required of discrete objects [Jo et al., 2022].

2.6 Discrete Diffusion

Considering these limitations, a growing line of work has adapted the diffusion framework to domains consisting of discrete variables such as text and graphs. In these settings, the central challenge is to define a noise process that converges to a simple prior and a corresponding reverse process that learns to remove the injected noise. Two leading approaches are DiGress [Vignac et al., 2022], and Masked Diffusion [Shi et al., 2024, Zhang et al., 2025].

2.6.1 DiGress

DiGress preserves the probabilistic structure of continuous diffusion, but adapts the underlying noise and transition mechanics to operate over discrete graph structures. DiGress corrupts graphs by applying independent edit operations such as adding, removing, or modifying edges, and then trains a graph transformer to predict these edits (Figure 2). This approach avoids the challenges of continuous noise and reformulates graph generation as a set of classification tasks [Yin et al., 2025].

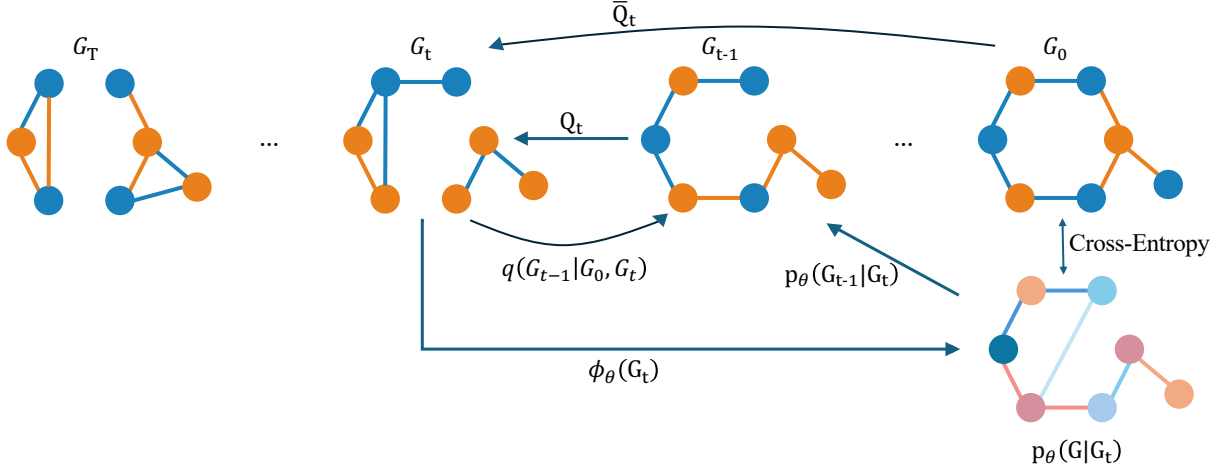


Figure 2: DiGress Diffusion as visualized in [Vignac et al., 2022]. The forward process is shown on top. Transitions from the clean graph G_0 to any timestep t are obtained using the cumulative transition matrix $\bar{Q}_t$. The reverse process is shown below. The true reverse distribution $q(G_{t-1} | G_0, G_t)$, depends on the unknown clean graph G_0 . A neural network is therefore trained to predict a surrogate distribution $p_\theta(G|G_t)$ over clean graphs. This prediction is combined with the known diffusion posterior to form $p_\theta(G_{t-1}|G_t)$ from which a graph at timestep $t - 1$ is sampled. The network is trained using a cross-entropy loss between the predicted clean graph distribution and the true clean graph.

Forward Process

Analogous to the continuous case, the DiGress forward process defines a Markov chain over graph states $G_t = (X_t, E_t)$, where X_t and E_t are respectively node and edge features. This process is expressed as

$$q(G_t | G_{t-1}) = \prod_i q(x_{t,i} | x_{t-1,i}) \prod_{i,j} q(e_{t,ij} | e_{t-1,ij}), \quad (14)$$

which gradually transforms the original graph G_0 into a fully randomized structure G_T . At convergence, this distribution approximates a factorized categorical prior:

$$q(G_T) \approx \prod_i \text{Cat}(x_{T,i}; \pi_X) \prod_{i,j} \text{Cat}(e_{T,ij}; \pi_E), \quad (15)$$

where π_X and π_E are uniform priors over node and edge types.

DiGress defines discrete noise transitions through categorical transition matrices that control the corruption level at each timestep. For each node (respectively edge), the transition probabilities are defined by the matrices:

$$[Q_t^X]_{ij} = q(x_t = j | x_{t-1} = i), \quad [Q_t^E]_{ij} = q(e_t = j | e_{t-1} = i), \quad (16)$$

where Q_t^X and Q_t^E specify the categorical transition probabilities between feature types at timestep t . The noised graph $G_t = (X_t, E_t)$ is then obtained by sampling each node and edge independently according to:

$$q(G_t | G_{t-1}) = (X_{t-1}Q_t^X, E_{t-1}Q_t^E). \quad (17)$$

However, as mentioned in the continuous diffusion section, in order not to have to simulate the entire Markov chain, it must be possible to sample any noisy state G_t directly from the clean graph. DiGress does this by using the product of all previous transition matrices $\bar{Q}_t = Q_1Q_2 \dots Q^t$. Sampling at any timestep is then done with:

$$q(G_t | G_0) = (X_0\bar{Q}_t^X, E_0\bar{Q}_t^E), \quad (18)$$

Reverse Process

The reverse diffusion process learns to denoise from the previous timestep by approximating the true posterior $q(G_{t-1} | G_t, G_0)$ with a neural network, $p_\theta(G_{t-1} | G_t)$. This posterior is computable in closed form because the forward process is a categorical Markov chain. For a single node or edge label z with one-hot encoding, Vignac et al. show that

$$q(z_{t-1} | z_t, z_0) \propto (z_t Q_t^\top) \odot (z_0 \bar{Q}_{t-1}), \quad (19)$$

where Q_t is the one-step transition matrix and $\odot$ denotes element-wise multiplication. Note that both terms $(z_t Q_t^\top)$ and $(z_0 \bar{Q}_{t-1})$ are probability vectors of the same dimension, ensuring the validity of the element-wise operation. Applying this independently to all node and edge attributes yields $q(G_{t-1} | G_t, G_0)$.

Just like in the continuous case, the clean graph G_0 is unknown at sampling time, so the exact posterior cannot be evaluated directly. DiGress addresses this by training the Graph Transformer ϕ_θ to predict the distribution of clean node and edge labels $p_\theta(z_0 | G_t)$ at every timestep. This prediction acts as a surrogate for the true clean graph in the posterior. The approximate reverse kernel is therefore obtained by marginalizing over the model’s clean-label prediction:

$$p_\theta(z_{t-1} | z_t) = \sum_{z_0} q(z_{t-1} | z_t, z_0) p_\theta(z_0 | G_t), \quad (20)$$

applied independently to all node and edge attributes. Sampling G_{t-1} from this categorical distribution yields a discrete denoising step that mirrors the continuous update.

In theory, DiGress maximizes a variational Evidence Lower Bound (ELBO) on $\log p_\theta(G_0)$ which contains KL divergences between the true discrete posterior and the model’s reverse distribution. However, since the forward process is categorical and produces one-hot labels, each KL term reduces to a standard cross-entropy loss. This yields the practical training objective:

$$L(\theta) = \sum_i \ell_X(p_\theta(x_i | G_t), x_i) + \sum_{i,j} \ell_E(p_\theta(e_{ij} | G_t), e_{ij}), \quad (21)$$

where x_i and e_{ij} are the true clean labels.

While DiGress represents a significant advance over continuous diffusion for graph generation, this formulation carries an inherent scalability limitation. Because both the forward and reverse processes must explicitly account for every possible node pair, even non-existent connections are modeled as a distinct “no-edge” type. Consequently, for a graph with n nodes, the model must maintain $\frac{n(n-1)}{2}$ edge representations - an inefficient overhead for sparse graphs where most edges are absent.

2.6.2 Masked Diffusion

Masked diffusion represents an alternative class of discrete diffusion models to DiGress, in which corruption is performed by replacing labels with a dedicated mask token, rather than replacing them with alternative values. Over time all labels are absorbed into the masked state, gradually driving the data toward an uninformative, fully masked prior. The reverse process then learns to reconstruct the masked elements conditioned on the unmasked context. Masked diffusion has shown promise in generating text ([Shi et al., 2024]), and more recently also been proposed to be applied to graphs ([Zhang et al., 2025]).

Forward Process

The forward process of masked diffusion is based on progressively masking the input representation, gradually replacing tokens with a special mask symbol according to a schedule of increasing corruption. Therefore, if there are m categories, an additional category, the masked state, is added to the representation [Shi et al., 2024]. Similarly to DiGress, one can express this corruption process using transition matrices that encode the noise schedule (see Eq. 16). However, rather than distributing probability over all possible categories, these matrices allocate mass only between the token remaining unchanged and flipping to the

mask state

$$Q_i = (1 - \beta_i) I + \beta_i \mathbf{1} e_m^\top, \quad (22)$$

where $\mathbf{1}$ is an all one vector of size $m + 1$ and e_m is a one-hot vector designating the masked type. β_i is the current noise level controlling the probability of corrupting a token into the mask state. To sample any noise state given the original data point x_0 is equivalent to Eq. 18. This process is *absorbing*, meaning that once a token is masked it can't change again. As the timestep t increases, the masking probability approaches 1, resulting in a prior where all tokens are masked.

Reverse Process

The reverse process in masked diffusion aims to recover samples from the data distribution by progressively unmasking tokens starting from a fully masked prior. Like other diffusion frameworks, the reverse process defines a Markov chain whose transitions are parameterized by a neural network trained to predict the original (clean) token distribution at masked positions.

The unmasking behavior is governed by a schedule α , which controls the probability that a token remains unmasked as a function of time. In the reverse process, this schedule determines how probability mass is transferred from the masked state back to clean token states.

For a token that is already unmasked at timestep t , the reverse posterior is trivial: the token must have had the same value at earlier timesteps, since masking is irreversible in the forward process. Consequently, the posterior assigns probability one to the current token value. If the token is masked at timestep t , however, there are two possible explanations: masking may have occurred prior to timestep $t - 1$, or it may have occurred during the transition from timestep $t - 1$ to t . This structure leads to the following posterior distribution:

$$q(x_{t-1} \mid x_t, x_0) = \text{Cat}(x_{t-1}; \bar{R}^{x_0}(t, t-1)^\top x_t). \quad (23)$$

Here $\bar{R}^{x_0}(t, t-1)$ is a transition matrix defined as

$$\bar{R}^{x_0}(t, t-1) = I + \frac{\alpha_{t-1} - \alpha_t}{1 - \alpha_t} e_m (x_0 - e_m)^\top \quad (24)$$

where α_t represents the cumulative probability schedule that a token remains unmasked at timestep t (replacing the generic noise level β_t used in other formulations). This matrix encodes the reverse transition

probabilities conditioned on the clean token x_0 . When applied to the one-hot representation of the current state x_t , it yields a categorical distribution over possible previous states, assigning probability mass either to the masked state or to the clean token, depending on the unmasking schedule defined by α_t .

As we have seen in other diffusion processes, the dependence of the posterior distribution on the clean label x_0 requires a model to estimate this clean label, as it is not available during inference. To address this, a model $p_\theta(x_0|x_t)$ is trained to predict a probability distribution over clean token types at masked positions using a cross entropy loss. In contrast to general discrete diffusion models, explicit marginalization over all possible clean labels is not required here, since the token cannot change its type throughout the forward diffusion. As a result, the predicted probability vector can be substituted directly into the posterior, producing a reverse transition distribution that depends only on x_t and from which x_{t-1} can be sampled.

The above formulation describes the reverse process for a single discrete token. For multidimensional data such as sequences or graphs, the forward diffusion process is chosen to factorize across tokens. Consequently, the forward marginals and reverse posteriors also factorize. The reverse model therefore processes the full corrupted input and is applied independently to each token.

3 Related Work

This section reviews MGDiff and SparseDiff, two recent graph diffusion models for molecular generation that are most relevant to this work. Both methods build on the DiGress framework, but they address different limitations: MGDiff focuses on improving the corruption process to better preserve chemically meaningful structure, while SparseDiff focuses on reducing memory complexity to enable scaling to larger graphs. Taken together, these approaches highlight two important directions in the development of diffusion-based graph generation and provide motivation for the proposed SCALE-MGD model.

3.1 MG-Diff

In 2025, [Zhang et al., 2025] introduced MG-Diff (Masked Graph Diffusion) a novel molecular graph diffusion model designed for both generation and optimization. The authors argue that uniform categorical corruption used in prior discrete graph diffusion models (e.g., DiGress-style replace-only noise) is too aggressive for molecular data, because it can abruptly replace atoms/bonds with semantically incompatible types and destabilize reverse denoising.

Instead, MG-Diff proposes a mask-and-replace diffusion strategy. It draws inspiration from both

Uniform Diffusion (2.6.1) and Masked Diffusion (2.6.2) to mitigate the issues that arise from each. Each token is either kept unchanged with probability α_t , replaced by a dedicated [MASK] token with a probability γ_t or uniformly replaced with small probability β_t . The masking component explicitly marks corrupted locations, so the denoiser can focus on reconstructing structure rather than first detecting where corruption occurred, while the small uniform-replacement component is retained to preserve editability: if a token is unmasked but wrong during sampling, pure mask-only diffusion would not provide a mechanism to revise it at later steps. Adding low-level uniform replacement allows the model to correct such errors during reverse diffusion. Although introducing uniform noise could theoretically destabilize convergence, the authors mitigate this by maintaining a sufficiently small β_t , ensuring that the denoising process remains stable while preserving editability.

Additionally, MG-Diff introduces a [PADDING] node label so molecules can be represented as a fixed-size graph with extra isolated placeholder nodes. During reverse diffusion, these placeholders can be activated into real atoms/bonds when needed, which supports variable-size generation.

Similarly to DiGress the graph is represented as $G = (A, C, E)$, where:

- $A \in R^{n \times d_A}$ is the node atom-type matrix;
- $C \in R^{n \times d_C}$ is the node charge matrix;
- $E \in R^{n \times n \times d_E}$ is the edge tensor.

In this work, the authors show a working mechanism of masked diffusion modeling with graphs. They are able to produce state of the art results and demonstrate higher validity scores compared to models such as DiGress model. However the edge tensor is dense with a categorical “no-bond” class, resulting in similar memory constraints as in DiGress. Furthermore, they use a graph transformer with global attention (see section 2.2.2).

3.2 SparseDiff

In 2025, [Qin et al., 2025] introduced SparseDiff, an adaptation of DiGress for larger graphs. They identified a memory complexity limitation in DiGress and modified its graph representation, noise model, and transformer architecture to reduce memory requirements.

The SparseDiff forward process is similar to that of DiGress. Noising transition matrices are iteratively applied to node and edge attributes to iteratively move their values from those of a clean graph

to the marginal distribution of those features in the dataset. SparseDiff represents edges with an edge list, where existing and non-existing edges are handled separately to improve space complexity. Any edges that transition to a non-existing state during the forward process are removed from the edge list. Meanwhile, a binomial distribution is sampled, and then that number of edges is sampled uniformly from the list of non-existing edges in the adjacency matrix.

SparseDiff’s reverse process is nearly identical to that of DiGress, except they divide the adjacency matrix into several sets of edges, which are predicted one-by-one at each time step t . Only the edges predicted to exist in each set are added to the noisy edge list for the next time step.

Memory complexity in DiGress primarily arises from the need to operate on the entire graph adjacency matrix during the forward process, leading to an $O(n^2)$ space complexity. To reduce the space complexity of the reverse process, SparseDiff introduces a sparsity parameter, denoted as λ . This parameter controls the fraction of non-existent edges that the model must evaluate at each step.

SparseDiff uses a sparse convolutional transformer model with local attention rather than global attention. At each time step, rather than evaluating all possible absent edges, a random set of query edges is sampled with a size proportional to λn^2 . The union of these query edges and the existing edges is fed into the model to enable a more global attention mechanism without materializing the full dense matrix. While this sparsity parameter λ successfully reduces the constant factor of the memory footprint, the number of query edges still inherently grows quadratically with respect to the number of nodes ($O(\lambda n^2)$). As a result, the fundamental scaling bottleneck remains at extreme graph sizes.

While SparseDiff significantly improves over DiGress’s space complexity, its noise model still involves an $O(n^2)$ term, due to sampling from the set of non-existent edges.

4 Methods

This section presents our approach to large sparse graph generation within the framework of graph diffusion models. We first introduce SCALE-MGD, an alternative modeling approach based on masked diffusion on a sparse representation of the graph. Secondly, we provide a more efficient version of the SparseDiff model, **SparserDiff**, that does not require to materialize the set of non-existing edges during the forward process.

The limitations identified in the preceding sections present three concrete obstacles to scaling graph diffusion. First, dense adjacency representations in DiGress and MGDiff require explicit modeling of all

$\frac{n(n-1)}{2}$ node pairs, regardless of how many edges actually exist. Second, the global attention mechanism in their Graph Transformer architectures compounds this with an additional $O(n^2)$ computational cost. Third, while SparseDiff avoids dense representations, its noise model still implicitly requires $\frac{n(n-1)}{2}$ space complexity when parameterizing the binomial distribution over non-existing edges. SCALE-MGD directly addresses the first two by operating masked diffusion on a sparse edge list with a GNN-based architecture. SparserDiff addresses the third by reformulating SparseDiff’s forward process to avoid materializing the set of non-existing edges entirely.

4.1 Scale-MGD

In this section, we propose a new method SCALE-MGD (Scalable Masked Graph Diffusion) that allows efficient large graph generation using sparse graph representations.

4.1.1 Sparse Graph Representation

Our graph representation $G = (X, E)$ consists of a list of node types (node features) and a list of edge tuples containing two node indices and the edge type. The edge list can be expressed as

$$E = [(i_0, j_0, \tau_0), (i_1, j_1, \tau_1), \dots, (i_m, j_m, \tau_m)] \quad (25)$$

Node indices are relative to the node-type list and are not absolute. In other words, edge endpoint indices are lookups in the node-type list.

We represent the graph using two token sequences:

- a node-type sequence $X \in \{1, \dots, K_X\}^n$
- an edge-tuple sequence $E \in (\mathcal{E} \cup \{\text{MASK}\})^m$

where $\mathcal{T}_X$ is the node token vocabulary size, including two special tokens: MASK and PAD. Each edge slot stores a tuple (i, j, τ) where $i, j \in \{1, \dots, n\}, \tau \in \mathcal{T}_{type}$ and $\mathcal{T}_{type}$ is the set of edge types.

During the forward noise process, entire edge tuples are masked or unmasked atomically: if an edge is selected for corruption, all three components (i, j, τ) are simultaneously replaced by MASK. However, the model *predicts* the three components independently via separate output heads, so the predicted distribution

factorizes as

$$p_{\theta}(e_k | x_t) = p_{\theta}(i_k | x_t) \cdot p_{\theta}(j_k | x_t) \cdot p_{\theta}(\tau_k | x_t). \quad (26)$$

The vocabularies for each component are: $\{1, \dots, n, \text{MASK}\}$ for each endpoint (size $n + 1$), and $\mathcal{T}_{type} \cup \{\text{NO_EDGE}, \text{MASK}\}$ for the edge type. MASK indicates that a node’s type is currently masked, while PAD represents a node that does not exist in our graph.

There are a fixed m number of edges in the edge list, where m is estimated from the maximum number of edges in the dataset. In the cases where $m < n^2$ and the graph is connected (the graph is relatively sparse), the space complexity of this representation is $O(m) < O(n^2)$.

In graphs with fewer than m edges, the remainder of the edges are padded as NO_EDGE types connecting to a padding (non-existent) node. Other options, like duplicating existing edges, were considered, but to avoid bias in predicting real edge tokens, we selected the NO_EDGE + PAD strategy.

4.1.2 Noise Model

This graph representation avoids the need for the noise model to sample from the adjacency matrix, but at the cost of requiring the model to learn to connect nodes via the fixed edge list.

Both DiGress and SparseDiff apply transition-matrix-based discrete noise to node and edge features, biasing transitions toward the dataset’s marginal distribution. To preserve the relationship between the edge list and the node-type list while simplifying the noise process, we adopt an absorbing (masked) diffusion approach in which tokens transition toward a special MASK state rather than toward marginal class frequencies. As discussed in 3.1, prior work on absorbing discrete diffusion (MG-Diff) has shown improved stability and competitive or superior validity metrics compared to marginal-transition-based discrete diffusion (i.e., DiGress and SparseDiff) on standard benchmarks.

Let the diffusion state at timestep t be

$$x_t = (X_t, E_t), \quad (27)$$

where X_t and E_t denote the node and edge tokens respectively. The forward diffusion process is governed by masked transition matrices. For node tokens we define

$$Q_t^{node} \in \mathbb{R}^{K_X \times K_X}. \quad (28)$$

The matrix has the form

$$Q_t^{node} = \begin{bmatrix} \alpha_t & \beta_t & \cdots & \beta_t & \gamma_t \\ \beta_t & \alpha_t & \cdots & \beta_t & \gamma_t \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \beta_t & \beta_t & \cdots & \alpha_t & \gamma_t \\ 0 & 0 & \cdots & 0 & 1 \end{bmatrix}, \quad (29)$$

where

$$(Q_t^{node})_{ij} = q(x_t = j \mid x_{t-1} = i), \quad (30)$$

and the last row and column correspond to the MASK state. Here α_t is the probability that a non-masked token remains unchanged, $\beta_t = (1 - \alpha_t - \gamma_t)/(K_X - 2)$ is the uniform probability of transitioning to any other non-masked token, and γ_t is the probability of becoming masked. Each row sums to one: $\alpha_t + (K_X - 2)\beta_t + \gamma_t = 1$. If a token is already masked (last row), it remains masked with probability 1.

These probabilities are determined by a linear masking schedule: the cumulative self-absorption $\bar{\alpha}_t$ is linearly interpolated from near 1 at $t = 1$ to near 0 at $t = T$, and the cumulative mask probability $\bar{\gamma}_t$ is interpolated inversely.

Edge tuples follow the same masked corruption process. The edge transition matrix has the same structure as Q_t^{node} , and is

$$Q_t^{edge} \in \mathbb{R}^{K_E \times K_E}. \quad (31)$$

As with nodes, each edge token either remains unchanged with probability α_t , transitions uniformly to another non-masked token with probability β_t , or becomes masked with probability γ_t . During both the forward and reverse processes the model predicts values for all tokens, so the output size remains $O(n + m)$.

4.1.3 Loss Calculation

We build on the masked discrete diffusion framework introduced by MG-Diff. Because node tokens and edge tokens are corrupted independently, the forward diffusion distribution factorizes as

$$q(x_t \mid x_0) = q(X_t \mid X_0)q(E_t \mid E_0). \quad (32)$$

For an individual token, the probability of observing x_t at timestep t given the original token x_0 is

$$q(x_t | x_0) = e_{x_t}^T \overline{\mathbf{Q}}_t e_{x_0}, \text{ where } \overline{\mathbf{Q}}_t = \mathbf{Q}_t \mathbf{Q}_{t-1} \cdots \mathbf{Q}_1. \quad (33)$$

MG-Diff optimizes the model using the variational lower bound

$$\begin{aligned} L_{\text{vlb}} = \mathbb{E}_{q(x_0)} \bigg[& \underbrace{\text{D}_{KL}[q(x_T | x_0) || p(x_T)]}_{L_T} + \sum_{t=2}^T \underbrace{\mathbb{E}_{q(x_t | x_0)} [\text{D}_{KL}[q(x_{t-1} | x_t, x_0) || p_\theta(x_{t-1} | x_t)]]}_{L_{t-1}} \\ & - \underbrace{\mathbb{E}_{q(x_t | x_0)} [\log p_\theta(x_0 | x_1)]}_{L_0} \bigg]. \end{aligned}$$

MG-Diff also introduces an auxiliary cross-entropy objective encouraging the model to predict the clean tokens x_0 :

$$L_{CE} = -\log p_\theta(x_0 | x_t). \quad (34)$$

For our representation this decomposes over node tokens and edge tokens:

$$L_{CE} = -\sum_{i=1}^n \log p_\theta(X_{0,i} | x_t) - \sum_{k=1}^m \log p_\theta(E_{0,k} | x_t). \quad (35)$$

The reverse transition distribution can be computed iteratively using

$$p_\theta(x_{t-1} | x_t) = \sum_{\tilde{x}_0} q(x_{t-1} | x_t, \tilde{x}_0) p_\theta(\tilde{x}_0 | x_t). \quad (36)$$

4.1.4 Neural Network Model

To avoid materializing the adjacency matrix (thereby avoiding an $O(n^2)$ space complexity), we utilize a GNN-based architecture, consisting of an encoder and decoder.

At the first time step of the reverse process, the graph has no information because it is completely masked. Even after the first few steps, the graph is still mostly masked, limiting the expressive power of a GNN.

To address this, we take inspiration from [Qin et al., 2025]’s SparseDiff for the encoder. Before the GNN layers run, the encoder randomly fills in the endpoints and edge types of masked edges based on their respective marginal distributions in the original dataset (Figure 3). This is analogous to the query edge

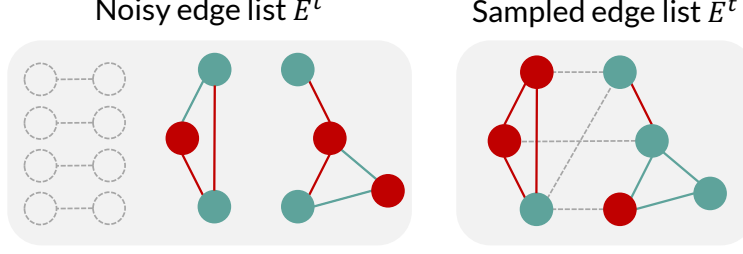


Figure 3: Before the noisy graph is fed to the GNN encoder, the masked edges are randomly populated according to the dataset marginal distributions for each edge tuple component.

system in SparseDiff, which they demonstrate to be more expressive than local message passing. Formally, with $\mathcal{K} = \{1, \dots, m\}$ our partially masked edge list at time t is:

$$E_t = \{e_k^t = (u_k^t, v_k^t, \tau_k^t) \mid \tau_k^t \neq \text{MASK}\}, \quad (37)$$

and we select edges $\mathcal{K}_{\text{mask}}^t$ to sample in the following way:

$$\mathcal{K}_{\text{mask}}^t = \mathcal{K} \setminus \mathcal{K}_{\text{obs}}^t \quad \text{where} \quad \mathcal{K}_{\text{obs}}^t = \{k \in \mathcal{K} \mid \tau_k^t \neq \text{MASK}\}. \quad (38)$$

For each edge indexed by k in $\mathcal{K}_{\text{mask}}^t$, we sample the endpoints and type independently:

$$\tilde{u}_k \sim \pi_u, \quad \tilde{v}_k \sim \pi_v, \quad \tilde{\tau}_k \sim \pi_\tau, \quad (39)$$

where π_u , π_v , and π_τ are the marginal distributions of source indices, target indices, and edge types (including the NO_EDGE type) computed from the training dataset, resulting in the message-passing edge list for timestep t :

$$\tilde{E}_t = E_t \cup \{(\tilde{u}_k, \tilde{v}_k, \tilde{\tau}_k) \mid k \in \mathcal{K}_{\text{mask}}^t\}. \quad (40)$$

The GNN decoder uses the encoder’s resulting node and edge embeddings to predict the clean node types, edge types, and endpoints. The space complexity of the output scales according to $O(n + m)$, since the graph representation remains at a fixed size throughout the model pass.

The full model forward pass is detailed in Algorithm 1. The encoder (Steps 3–8) runs L_{enc} independent random-unmasking trials: in each trial, every MASK slot in the edge list has its source, target, and type independently replaced by samples from the marginal distributions π_u , π_v , π_τ . A stack of L_{mp} GNN message-passing layers with residual connections then produces node embeddings, which are averaged across

all trials. The decoder (Steps 10–13) constructs per-edge-slot embeddings by summing embedding lookups for the source index, target index, edge type, a positional encoding, and the sinusoidal timestep embedding. A virtual zero-vector node is appended at index $n + 1$ so that edges whose endpoints are still MASK gather a neutral embedding. Each of the L_{dec} decoder layers updates edge embeddings via an MLP conditioned on the gathered endpoint node embeddings, operating in $O(m)$ time. Four linear heads produce logits for node types, edge source pointers, edge target pointers, and edge types.

Algorithm 2 describes one VLB training step. A random timestep is sampled, the clean graph is corrupted via the forward process (with atomic masking of edge tuples), and the model predicts the clean-token distribution. The reverse posterior is computed in closed form by passing the predicted distribution through Bayes’ rule in log-space, and the loss combines an importance-weighted KL divergence with an adaptive auxiliary cross-entropy term.

Algorithm 3 describes ancestral sampling. Starting from a fully masked state, the model iteratively predicts the clean distribution, computes temperature-scaled posteriors, and samples the next less-noisy state. After each step, atomic mode enforcement ensures that if an edge’s type component is still MASK, its endpoint components remain masked as well, and endpoint ordering $u < v$ is enforced for non-masked edges.

Algorithm 1 GNN Model $f_\theta(\mathbf{X}_t, \mathbf{E}_t, t)$

Require: Noisy graph $(\mathbf{X}_t, \mathbf{E}_t)$, timestep t , marginal priors π_u, π_v, π_τ , L_{enc} encoder iterations, L_{mp} message-passing layers per iteration, L_{dec} decoder layers

```
1:  $\mathbf{t}_{\text{emb}} \leftarrow \text{SINUSOIDALEMBED}(t)$ 
2:  $\mathbf{H}_X \leftarrow \mathbf{0}^{n \times H}$ 
3: for  $\ell = 1, \dots, L_{\text{enc}}$  do  $\triangleright L_{\text{enc}}$  independent random-unmasking encoder trials
4:    $\tilde{\mathbf{E}} \leftarrow \text{RANDOMUNMASK}(\mathbf{E}_t, \pi_u, \pi_v, \pi_\tau)$   $\triangleright$  For each MASK component in  $\mathbf{E}_t$ , sample  $u \sim \pi_u, v \sim \pi_v,$   
 $\tau \sim \pi_\tau$  independently
5:    $\mathbf{h} \leftarrow \text{GNNENCODER}(\mathbf{X}_t, \tilde{\mathbf{E}}, \mathbf{t}_{\text{emb}})$   $\triangleright L_{\text{mp}}$  message-passing layers with residual connections
6:    $\mathbf{H}_X \leftarrow \mathbf{H}_X + \mathbf{h}$ 
7: end for
8:  $\mathbf{H}_X \leftarrow \mathbf{H}_X / L_{\text{enc}}$   $\triangleright$  Mean-pool over trials to form final node embeddings
9: for  $k = 1, \dots, m$  do  $\triangleright$  Embed each edge slot from its endpoints, type, slot position, and timestep
10:    $\mathbf{h}_e^{(k)} \leftarrow \mathbf{W}_u[u_k] + \mathbf{W}_v[v_k] + \mathbf{W}_\tau[\tau_k] + \mathbf{PE}(k) + \mathbf{t}_{\text{emb}}$   $\triangleright [\cdot]$  denotes embedding lookup;  
    $u_k, v_k \in \{1, \dots, n, \text{MASK}\}, \tau_k \in \mathcal{T}_{\text{type}} \cup \{\text{MASK}\}$ 
11: end for
12:  $\hat{\mathbf{H}}_X \leftarrow [\mathbf{H}_X \parallel \mathbf{0}]$   $\triangleright$  Append virtual zero-vector node at index  $n+1$  so MASK pointers gather a neutral  
embedding
13: for  $\ell = 1, \dots, L_{\text{dec}}$  do  $\triangleright L_{\text{dec}}$  GNN decoder layers, each  $O(m)$ 
14:   for  $k = 1, \dots, m$  do
15:      $\mathbf{h}_e^{(k)} \leftarrow \mathbf{h}_e^{(k)} + \text{MLP}([\mathbf{h}_e^{(k)}; \hat{\mathbf{H}}_X[u_k]; \hat{\mathbf{H}}_X[v_k]])$ 
16:   end for
17: end for
18:  $(\hat{\ell}_X, \hat{\ell}_u, \hat{\ell}_v, \hat{\ell}_\tau) \leftarrow (\mathbf{W}^X \mathbf{H}_X, \mathbf{W}^u \mathbf{h}_e, \mathbf{W}^v \mathbf{h}_e, \mathbf{W}^\tau \mathbf{h}_e)$   $\triangleright$  Four linear heads producing logits for node types,  
source/target pointers, and edge types
19: return  $(\hat{\ell}_X, \hat{\ell}_u, \hat{\ell}_v, \hat{\ell}_\tau)$ 
```

4.1.5 Training

Algorithm 2 VLB training at timestep t

Require: Clean graph $G_0 = (\mathbf{X}_0, \mathbf{E}_0)$; absorbing schedule $\{\bar{\alpha}_t^{(c)}, \bar{\beta}_t^{(c)}, \bar{\gamma}_t^{(c)}\}$ per modality $c \in \{X, u, v, \tau\}$;
 auxiliary loss weight λ_{aux}

- 1: Sample $t \sim \text{Uniform}\{1, \dots, T\}$
- 2: $x_t^{(c)} \sim q(x_t^{(c)} | x_0^{(c)}) = \bar{\alpha}_t^{(c)} \mathbf{1}[x_t^{(c)} = x_0^{(c)}] + \bar{\beta}_t^{(c)} + \bar{\gamma}_t^{(c)} \mathbf{1}[x_t^{(c)} = \text{MASK}] \quad \forall c$ ▷ Sample noisy graph; in atomic-edge mode, if any edge component is masked, all three (u, v, τ) are masked jointly
- 3: $(\hat{\ell}_X, \hat{\ell}_u, \hat{\ell}_v, \hat{\ell}_\tau) = f_\theta(\mathbf{X}_t, \mathbf{E}_t, t)$ ▷ Predict clean-graph distribution (Algorithm 1)
- 4: $p_\theta(x_0^{(c)} | x_t) \leftarrow \text{softmax}(\hat{\ell}^{(c)}) \quad \forall c$ ▷ Convert logits to predicted clean-token probabilities
- 5: $\log p_\theta(x_{t-1}^{(c)} | x_t^{(c)}) = \log \sum_{x'_0} q(x_{t-1}^{(c)} | x_t^{(c)}, x'_0) p_\theta(x_0^{(c)} = x'_0 | x_t^{(c)}) \quad \forall c$ ▷ Vectorized logsumexp over all K classes; no explicit loop over x'_0
- 6: $\mathcal{L}_{\text{VLB}} = T \cdot \sum_c \text{KL}[q(x_{t-1}^{(c)} | x_t^{(c)}, x_0^{(c)}) \parallel p_\theta(x_{t-1}^{(c)} | x_t^{(c)})]$ ▷ Importance-weighted ($\times T$) KL at $t > 0$; decoder NLL at $t = 0$
- 7: $\mathcal{L}_{\text{aux}} = \lambda_{\text{aux}} \cdot (1 + \frac{T-t}{T}) \cdot T \cdot \sum_c \text{KL}[x_0^{(c)} \parallel p_\theta(x_0^{(c)} | x_t^{(c)})]$ ▷ Adaptive auxiliary loss: weight ranges from 2 at $t=0$ to 1 at $t=T$, encouraging direct x_0 prediction
- 8: **optimizer.step**($\mathcal{L}_{\text{VLB}} + \mathcal{L}_{\text{aux}}$)

4.1.6 Sampling

Algorithm 3 Ancestral sampling

Require: Trained f_θ (Algorithm 1); schedule $\{\bar{\alpha}_t^{(c)}, \bar{\beta}_t^{(c)}, \bar{\gamma}_t^{(c)}\}$; temperature σ

- 1: $\mathbf{X}_T \leftarrow \text{MASK}^n$; $\mathbf{E}_T \leftarrow (\text{MASK}, \text{MASK}, \text{MASK})^m$ ▷ Initialise all node and edge slots as MASK
- 2: **for** $t = T, T-1, \dots, 1$ **do**
- 3: $(\hat{\ell}_X, \hat{\ell}_u, \hat{\ell}_v, \hat{\ell}_\tau) = f_\theta(\mathbf{X}_t, \mathbf{E}_t, t)$ ▷ Predict clean-graph logits from current noisy state
- 4: $p_\theta(x_0^{(c)} | x_t) \leftarrow \text{softmax}(\hat{\ell}^{(c)}/\sigma) \quad \forall c$ ▷ Apply temperature σ to logits before softmax
- 5: $\log p_\theta(x_{t-1}^{(c)} | x_t^{(c)}) = \log \sum_{x'_0} q(x_{t-1}^{(c)} | x_t^{(c)}, x'_0) p_\theta(x_0^{(c)} = x'_0 | x_t^{(c)}) \quad \forall c$ ▷ Compute posterior via Bayes' rule in log-space
- 6: $x_{t-1}^{(c)} \sim p_\theta(x_{t-1}^{(c)} | x_t^{(c)}) \quad \forall c$
- 7: **if** edge type τ_k^t is MASK for edge k **then**
- 8: Keep $u_k^{t-1}, v_k^{t-1}, \tau_k^{t-1} \leftarrow \text{MASK}$ ▷ Atomic mode: do not partially unmask edges
- 9: **end if**
- 10: Enforce $u_k < v_k$ for all non-masked edges by swapping if needed
- 11: **end for**
- 12: **return** $(\mathbf{X}_0, \mathbf{E}_0)$

4.2 SparserDiff: A Memory-Efficient Variant of SparseDiff

In SparseDiff, the authors address many aspects of DiGress with quadratic space complexity by modifying the neural network structure, noise model, and sampling processes [Qin et al., 2025]. While many of these solutions achieve sub-quadratic space complexity, the noise model they present has a dominant $O(n^2)$ term. This space complexity arises from their new edge sampling step, which samples nonexistent edges and adds them to the graph. According to the authors, the largest contributor to this space complexity is sampling new edge positions.

We introduce an alternative algorithm with worst-case space complexity of $O(n + m + k)$, where k is the number of sampled edges. Given an edge list E and the number of edges to sample k , we first construct sorted adjacency lists. Each edge appears only once in this adjacency list structure as (u, v) , where $u < v$.

We precompute the number of missing edges for each node u (array H), and then compute the prefix array L containing the number of missing edges prior to node u . From the total number of missing edges, we sample k indices without replacement using Floyd’s optimal sampling algorithm. To sample each edge based on the sampled index, we first use binary search in L to find the node u that this edge will connect to and the column index j of the edge (the index the edge would have in an adjacency matrix). We then construct f , an array storing the number of missing edges before each edge in node u ’s adjacency list.

We again use binary search to find the insertion point ip in node u ’s adjacency list, and compute the second node in the sampled edge with $v = (u + 1) + j + ip$. Decomposing this, we start with $(u + 1)$, which is the first possible edge index for node v , considering node u . We add j , the desired index of the edge, and ip , the number of existing edges that appear before reaching j in the adjacency list of u . We repeat this procedure for sampling all edges. Figure 4 shows a toy example to illustrate the algorithm.

4.2.1 Space Complexity Analysis

The space complexity of the algorithm is as follows:

1. Adjacency lists: $O(n + m)$
2. Compute H, L : $O(n)$
3. Sampled edge indices: $O(k)$
4. Cache f for each node: $O(m)$

In the worst case, the space complexity is $O(n + m + k)$.

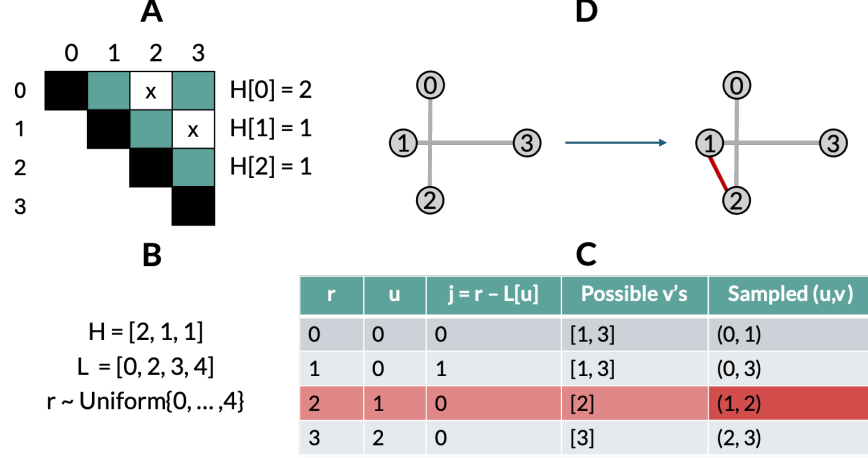


Figure 4: Nonexistent edges in the adjacency matrix (A) are sampled by computing a prefix array of the number of missing edges before each node and sampling indices of nonexistent edges (B) and utilizing binary search to identify the real edge corresponding to each sampled index (C), leading to a set of new sampled edges (D).

4.2.2 Time Complexity Analysis

The time complexity of the algorithm is as follows:

1. Build adjacency lists: $O(m)$
2. Sort adjacency lists: $O\left(\sum_{u=0}^{n-1} d_u \log d_u\right)$, where $d_u = |\text{adj}[u]|$
3. Compute H, L : $O(n)$
4. Sample k indices: $O(k)$
5. Map sampled indices to edges: $O(\log n + \sum_u d_u)$
 - (a) Find row u with binary search on L : $O(\log n)$
 - (b) Compute column index j : $O(1)$
 - (c) Build f for row u if not cached: $O(d_u)$
 - (d) Binary search in f (length d_u): $O(\log d_u)$
 - (e) Compute second edge endpoint v : $O(1)$

In the worst case, the time complexity is $O(m + \sum_u d_u \log d_u + n + k \log n)$

Algorithm 4 Space-Efficient Edge Sampling(E, n, k)

Require: Edge list E , number of nodes n , number of samples k

Ensure: Sampled missing edges $\mathcal{S} \subseteq \{(u, v) \mid 0 \leq u < v < n\}$

```
1: Build upper-triangular adjacency lists  $\text{adj}[u]$  from  $E$  (drop self-loops, enforce  $u < v$ ), then sort each
    $\text{adj}[u]$ 
2:  $\text{running} \leftarrow 0$ 
3: for  $u = 0$  to  $n - 1$  do
4:    $H[u] \leftarrow (n - 1 - u) - |\text{adj}[u]|$ 
5:    $L[u] \leftarrow \text{running}$ 
6:    $\text{running} \leftarrow \text{running} + H[u]$ 
7: end for
8:  $T \leftarrow \text{running}$  ▷ total missing edges
9: if  $T < k$  then
10:   error
11: end if
12:  $R \leftarrow \text{FLOYDSAMPLEWITHOUTREPLACEMENT}(T, k)$ 
13:  $\mathcal{S} \leftarrow []$ ,  $C \leftarrow$  empty map ▷  $C[u]$  caches transformed gaps
14: for all  $\rho \in R$  do
15:    $u \leftarrow \text{BISECTRIGHT}(L, \rho) - 1$ 
16:    $j \leftarrow \rho - L[u]$ 
17:   if  $u \notin C$  then
18:      $C[u] \leftarrow []$ 
19:     for  $i = 0$  to  $|\text{adj}[u]| - 1$  do
20:        $\text{gap} \leftarrow \text{adj}[u][i] - (u + 1) - i$ 
21:       append  $\text{gap}$  to  $C[u]$ 
22:     end for
23:   end if
24:    $\text{ip} \leftarrow \text{BISECTRIGHT}(C[u], j)$ 
25:    $v \leftarrow (u + 1) + j + \text{ip}$ 
26:   append  $(u, v)$  to  $\mathcal{S}$ 
27: end for
28: return  $\mathcal{S}$ 
```

5 Experiments and Results

In this section, we present a comprehensive evaluation of our proposed methods, SparserDiff and SCALE-MGD. We begin in Section 5.1 with a macro-level profiling study that evaluates the memory and time scaling behavior of both our methods against existing baselines under controlled synthetic conditions. Following this overarching complexity analysis, we provide deep-dive evaluations specific to each proposed architecture. Section 5.2 focuses on SparserDiff, presenting algorithmic microbenchmarks of its novel edge-sampling routine alongside its downstream generation validity on molecular and social graphs. Finally, Section 5.3 evaluates SCALE-MGD, detailing its unique performance characteristics and generative capabilities. This hybrid structure allows us to first establish the fundamental scaling advantages of our approaches globally, before dissecting their individual correctness, performance trade-offs, and generation quality in detail.

5.1 Evaluating Memory and Time Scaling

5.1.1 Experiment Settings

We evaluate the theoretical time and space complexity of our two methods, SCALE-MGD and SparserDiff, and compare them with three existing algorithms DiGress, SparseDiff, and MG-Diff [Vignac et al., 2022, Qin et al., 2025, Zhang et al., 2025]. Table 1 shows the hyperparameters used for the neural network backbones¹ in this profiling study.

Table 1: Hyperparameters of the neural networks used in the complexity analysis.

Hyperparameter	Value	Description
n_{layers}	5	Number of Transformer layers
att_{heads}	8	Number of attention heads
d_x	256	Hidden node feature dimension
d_e	64	Hidden edge feature dimension
d_y	64	Hidden global feature dimension
d_{ff}	256, 64, 64	Feed-forward dims (Node, Edge, Global)
d_{mlp}	128, 64, 128	MLP hidden dims (Node, Edge, Global)

To gain fine control over the graph size, rather than using graphs from an existing dataset, we uniformly generate synthetic inputs in compliance with requirements of different algorithms. Specifically, we

¹The authors that introduced MG-Diff did not provide an implementation of the neural network, so we implemented a Graph Transformer for MG-Diff from scratch based on their supplementary text at <https://doi.org/10.1371/journal.pone.0331450.s001>.

use the **Erdős–Rényi** $G(N, M)$ graph model to create graphs with a prescribed number of nodes N and edges M . For all configurations, we generate undirected graphs without self-loops and set $M = \min(5N, \lfloor N(N - 1)/2 \rfloor)$ to ensure an average degree ≤ 5 , consistent with the sparse graphs we target. For node features, we generate one-hot tensors of dimensions (N, d_X) . For SparserDiff and the baselines, which operate on full adjacency matrices, we generate tensors of (N, N, d_E) representing all possible edges and their one-hot types. For SCALE-MGD, we generate edge list tensors of dimensions $(M, 3)$, where each row stores (Source, Target, Edge Type). In addition to varying the size of the graph, we isolate the effect of increased feature dimensions by fixing the graph size at $N = 50$ for two configurations. Table 2 specifies the types of graphs targeted in the evaluation, including the number of edges per graph derived from the $G(N, M)$ model.

Table 2: Graph type specifications used in the experiments (batch size = 128 for all).

Graph Type	Nodes (N)	Edges (M)	d_X	d_E
Molecular Graph	20	100	9	4
Small Social Network	50	250	9	4
Medium Graph	100	500	9	4
Large Graph	200	1000	9	4
Complex Edge Types	50	250	9	8
Complex Node Types	50	250	20	4

For SparserDiff and SparseDiff, which employ a subsampling strategy during training, we set the edge fraction hyperparameter to 0.1. This parameter controls the size of the subgraph used in each training step (setting it to 1 would correspond to using the full adjacency matrix). The choice of 0.1 aligns with the sparse graph regime we target and ensures computational efficiency while maintaining representative graph structure.

For each graph type, we measure the wall-clock time, memory usage, and GPU RAM consumption for each algorithm during four stages: **forward diffusion**, **model pass**, **training step**², and **sampling**. We ran each algorithm on 10 different random seeds to account for randomness and aggregated the results. To speed up the experiment and allow for a consistent comparison, we test and report the results for diffusion process standardized to 10 steps in **forward diffusion** and **sampling**, in contrast to 500 steps which is used as default in all four algorithms.

All experiments were performed on the WPI Turing cluster equipped with 4 CPU cores, 32GB of memory, and an A100-80G GPU.

²A training step is composed with: noise addition (one step), model pass, and loss computation.

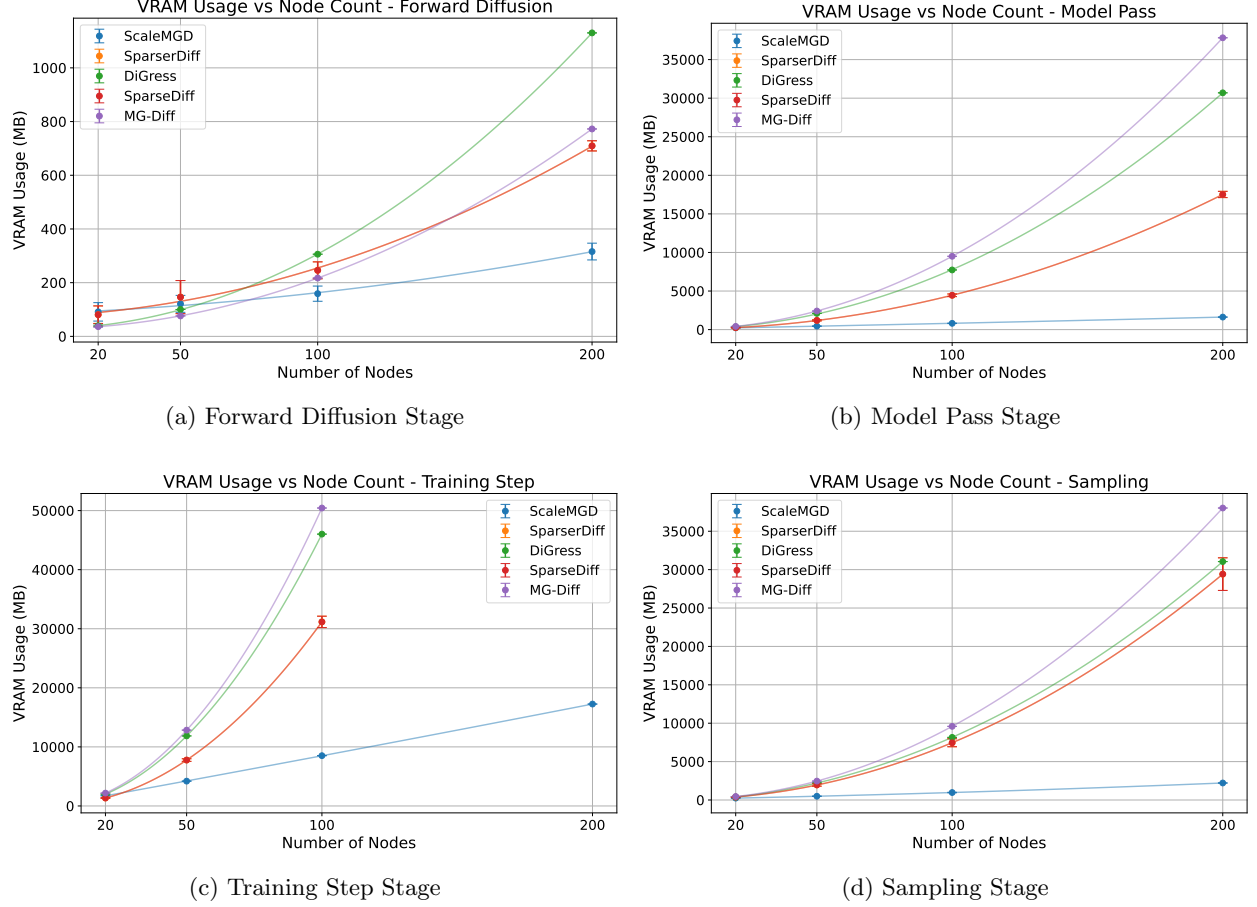


Figure 5: VRAM usage comparison across different algorithms and stages with respect to the number of nodes on synthetic graphs. Trends indicate the space complexity of each method.

5.1.2 Results and Discussion

To visualize the scaling behavior of each method, we plot the average GPU VRAM consumption in different stages with respect to the number of nodes in Figure 5 using results from the first four graph types (i.e. **Molecular Graph**, **Small Social Network**, **Medium Graph**, **Large Graph**). We observe a significant advantage of SCALE-MGD in terms of memory efficiency, as it is the only algorithm capable of scaling to the **Large Graph** configuration with $N = 200$ without exhausting the 80GB GPU memory, maintaining the lowest VRAM usage during both the training and sampling steps. We also note that the memory consumption of SCALE-MGD grows significantly slower than the baselines, aligning with its theoretical linear space complexity. In contrast, SparserDiff and the three baseline algorithms exhibit quadratic growth in VRAM usage, demonstrating $O(N^2)$ space complexity.

We show the full results in Tables 3–8. In the following paragraphs, we highlight several key observations.

Quadratic scaling limits baseline methods. The results confirm the theoretical $O(N = n^2)$ space complexity of the baseline algorithms. For $N = 50$ (Table 4), DiGress and MG-Diff already require over 11GB and 12GB of GPU memory during training respectively, while SCALE-MGD only requires approximately 4GB. For $N = 100$ (Table 5), training memory for DiGress (46GB) and MG-Diff (50GB) further exceeds SCALE-MGD’s memory usage (8.5GB) by a factor of 5–6 $\times$. SparseDiff, despite its sparse optimizations, still consumes approximately 40GB at this scale, confirming that quadratic complexity eventually dominates its complexity. SparserDiff, which shares the same quadratic input representation, exhibits nearly identical memory usage to SparseDiff at smaller scales (e.g., $N = 20$ and $N = 50$) and similarly fails to scale to $N = 200$ due to out-of-memory errors.

Linear scaling enables large graphs. The power of linear scaling becomes evident at $N = 200$ (Table 6). All other algorithms fail during training due to out-of-memory errors, while SCALE-MGD completes all stages using only 17GB GPU memory out of the capacity of 80GB. This demonstrates that only SCALE-MGD provides a viable approach for scaling graph diffusion models to large graphs where dense matrix operations become computation expensive.

Robustness to complex features. Tables 7 and 8 show that increasing edge feature dimensions ($d_E = 8$) or node feature dimensions ($d_X = 20$) does not alter the relative performance. SCALE-MGD maintains its memory advantage, while the baselines continue to exhibit the same quadratic scaling patterns observed in the standard graph configurations. We also note that SparserDiff is able to outperform the original SparseDiff by a large margin in the forward diffusion stage.

Training-inference asymmetry in subsampling-based methods. SparserDiff and SparseDiff use a subsampling strategy during training with an edge fraction of 0.1, which reduces computational cost compared to full-graph methods like DiGress. At $N = 50$ (Table 4), this advantage is evident: both methods complete the model pass in 0.084s and consume 7.8GB GPU memory during training, outperforming DiGress (0.102s, 11.9GB) and MG-Diff (0.123s, 12.8GB). However, this benefit does not extend to full graph generation, where subsampling introduces overhead, leading to substantially longer sampling times (SparserDiff 12s vs. DiGress 1.1s). Notably, SparserDiff still maintains a memory advantage during sampling (1.96GB vs. 2.22GB), indicating that the primary cost is increased inference time rather than memory footprint.

Time-space trade-offs. While SCALE-MGD shows dominant performance in memory efficiency, its wall-clock time is consistently higher than the baselines, particularly during the model pass and training step. This reflects the overhead of the random completion process in edge list representation. However, the gap narrows as graph size increases. Notably, SCALE-MGD’s training step time increases only modestly with graph size (from 2.47s at $N = 20$ to 2.91s at $N = 200$), whereas baseline methods either exhibit substantial growth or fail entirely due to memory exhaustion. This stable time scaling, combined with its linear memory footprint, makes SCALE-MGD a viable option for large graphs where memory constraints would otherwise prevent training.

Table 3: Molecular Graph ($N = 20$)

Stages	Models	Time (s)	GPU (MB)	RAM (MB)
Forward Diffusion	Scale-MGD	0.0004 $\pm$ 0.0001	91.26 $\pm$ 34.33	1108.68 $\pm$ 58.19
	SparsersDiff	0.0038 $\pm$ 0.0002	80.76 $\pm$ 32.66	2144.99 $\pm$ 10.86
	DiGress	0.0017 $\pm$ 0.0001	39.52 $\pm$ 2.45	1271.12 $\pm$ 34.75
	SparseDiff	0.0047 $\pm$ 0.0002	80.76 $\pm$ 32.66	2156.02 $\pm$ 11.12
	MG-Diff	0.0022 $\pm$ 0.0001	36.23 $\pm$ 2.44	1370.20 $\pm$ 60.97
Model Pass	Scale-MGD	0.9386 $\pm$ 0.0020	225.60 $\pm$ 34.35	1108.70 $\pm$ 58.14
	SparsersDiff	0.0217 $\pm$ 0.0007	254.32 $\pm$ 32.99	2144.99 $\pm$ 10.86
	DiGress	0.0304 $\pm$ 0.0247	349.98 $\pm$ 2.45	1278.55 $\pm$ 12.47
	SparseDiff	0.0217 $\pm$ 0.0007	254.32 $\pm$ 32.99	2156.02 $\pm$ 11.12
	MG-Diff	0.0261 $\pm$ 0.0006	416.04 $\pm$ 2.44	1370.21 $\pm$ 60.96
Training Step	Scale-MGD	2.4667 $\pm$ 0.0832	1729.68 $\pm$ 30.43	1125.68 $\pm$ 8.44
	SparsersDiff	0.0600 $\pm$ 0.0066	1344.69 $\pm$ 56.79	2145.20 $\pm$ 10.45
	DiGress	0.0477 $\pm$ 0.0118	1999.52 $\pm$ 0.26	1281.60 $\pm$ 3.51
	SparseDiff	0.0689 $\pm$ 0.0245	1344.69 $\pm$ 56.79	2156.23 $\pm$ 10.70
	MG-Diff	0.0885 $\pm$ 0.0328	2179.29 $\pm$ 2.40	1390.29 $\pm$ 0.76
Sampling	Scale-MGD	19.0962 $\pm$ 0.0359	244.22 $\pm$ 30.65	1126.08 $\pm$ 7.31
	SparsersDiff	2.7033 $\pm$ 0.1940	387.11 $\pm$ 33.20	2145.99 $\pm$ 9.62
	DiGress	0.2544 $\pm$ 0.0056	386.50 $\pm$ 0.41	1283.70 $\pm$ 1.67
	SparseDiff	2.6642 $\pm$ 0.1404	387.13 $\pm$ 33.22	2157.15 $\pm$ 10.20
	MG-Diff	0.2925 $\pm$ 0.0039	455.73 $\pm$ 0.00	1390.38 $\pm$ 0.62

Table 4: Small Social Network ($N = 50$)

Stages	Models	Time (s)	GPU (MB)	RAM (MB)
Forward Diffusion	Scale-MGD	0.0004 $\pm$ 0.0001	121.08 $\pm$ 31.57	1127.66 $\pm$ 2.26
	SparsersDiff	0.0055 $\pm$ 0.0016	146.28 $\pm$ 61.66	2173.74 $\pm$ 5.83
	DiGress	0.0019 $\pm$ 0.0001	100.20 $\pm$ 0.05	1283.53 $\pm$ 0.07
	SparseDiff	0.0055 $\pm$ 0.0003	146.28 $\pm$ 61.66	2188.35 $\pm$ 6.87
	MG-Diff	0.0020 $\pm$ 0.0000	76.10 $\pm$ 0.00	1394.17 $\pm$ 2.78
Model Pass	Scale-MGD	0.9438 $\pm$ 0.0054	449.48 $\pm$ 31.57	1127.66 $\pm$ 2.26
	SparsersDiff	0.0837 $\pm$ 0.0027	1195.84 $\pm$ 74.00	2173.74 $\pm$ 5.83
	DiGress	0.1022 $\pm$ 0.0012	2100.76 $\pm$ 0.05	1283.53 $\pm$ 0.07
	SparseDiff	0.0837 $\pm$ 0.0022	1195.84 $\pm$ 74.00	2188.37 $\pm$ 6.87
	MG-Diff	0.1227 $\pm$ 0.0005	2411.12 $\pm$ 0.00	1394.17 $\pm$ 2.78
Training Step	Scale-MGD	2.7468 $\pm$ 0.0398	4203.16 $\pm$ 31.42	1127.97 $\pm$ 2.22
	SparsersDiff	0.2266 $\pm$ 0.0136	7770.98 $\pm$ 217.27	2173.75 $\pm$ 5.83
	DiGress	0.1246 $\pm$ 0.0013	11856.00 $\pm$ 0.06	1283.54 $\pm$ 0.07
	SparseDiff	0.2241 $\pm$ 0.0050	7770.98 $\pm$ 217.27	2188.38 $\pm$ 6.87
	MG-Diff	0.2540 $\pm$ 0.0003	12839.38 $\pm$ 0.00	1394.18 $\pm$ 2.78
Sampling	Scale-MGD	19.3100 $\pm$ 0.0293	489.58 $\pm$ 31.23	1127.97 $\pm$ 2.22
	SparsersDiff	12.0645 $\pm$ 1.0413	1960.51 $\pm$ 157.93	2175.49 $\pm$ 1.04
	DiGress	1.0871 $\pm$ 0.0116	2223.61 $\pm$ 0.10	1290.72 $\pm$ 0.13
	SparseDiff	12.0197 $\pm$ 1.0201	1960.50 $\pm$ 157.58	2190.50 $\pm$ 2.82
	MG-Diff	1.2539 $\pm$ 0.0018	2461.61 $\pm$ 0.00	1394.41 $\pm$ 2.83

Table 5: Medium Sized Graph ($N = 100$)

Stages	Models	Time (s)	GPU (MB)	RAM (MB)
Forward Diffusion	Scale-MGD	0.0004 $\pm$ 0.0001	159.04 $\pm$ 28.32	1127.37 $\pm$ 0.18
	SparsersDiff	0.0075 $\pm$ 0.0006	246.02 $\pm$ 31.60	2195.79 $\pm$ 8.23
	DiGress	0.0054 $\pm$ 0.0001	306.05 $\pm$ 0.05	1299.72 $\pm$ 9.92
	SparseDiff	0.0066 $\pm$ 0.0003	245.88 $\pm$ 31.48	2201.85 $\pm$ 4.61
	MG-Diff	0.0029 $\pm$ 0.0001	217.31 $\pm$ 0.00	1401.48 $\pm$ 3.56
Model Pass	Scale-MGD	0.9388 $\pm$ 0.0021	814.13 $\pm$ 28.51	1127.37 $\pm$ 0.18
	SparsersDiff	0.3690 $\pm$ 0.0149	4447.99 $\pm$ 171.20	2195.79 $\pm$ 8.23
	DiGress	0.3854 $\pm$ 0.0053	7733.94 $\pm$ 0.05	1299.73 $\pm$ 9.90
	SparseDiff	0.3704 $\pm$ 0.0148	4448.06 $\pm$ 171.35	2201.85 $\pm$ 4.61
	MG-Diff	0.4628 $\pm$ 0.0003	9505.76 $\pm$ 0.28	1401.48 $\pm$ 3.56
Training Step	Scale-MGD	2.8245 $\pm$ 0.0474	8512.84 $\pm$ 30.24	1127.49 $\pm$ 0.14
	SparsersDiff	0.9085 $\pm$ 0.0203	31159.69 $\pm$ 965.76	2195.80 $\pm$ 8.22
	DiGress	0.4170 $\pm$ 0.0053	46018.20 $\pm$ 0.05	1299.78 $\pm$ 9.91
	SparseDiff	0.9083 $\pm$ 0.0211	31159.14 $\pm$ 965.36	2201.86 $\pm$ 4.60
	MG-Diff	0.8908 $\pm$ 0.0006	50437.02 $\pm$ 0.29	1401.50 $\pm$ 3.54
Sampling	Scale-MGD	19.5365 $\pm$ 0.0428	972.45 $\pm$ 30.18	1127.49 $\pm$ 0.14
	SparsersDiff	55.3523 $\pm$ 5.3280	7456.17 $\pm$ 520.74	2198.35 $\pm$ 5.23
	DiGress	4.0578 $\pm$ 0.0535	8155.32 $\pm$ 0.05	1303.37 $\pm$ 11.22
	SparseDiff	55.3162 $\pm$ 5.4069	7456.40 $\pm$ 520.29	2203.51 $\pm$ 4.17
	MG-Diff	4.6582 $\pm$ 0.0033	9592.29 $\pm$ 0.28	1402.46 $\pm$ 0.64

Table 6: Large Graph ($N = 200$)

Stages	Models	Time (s)	GPU (MB)	RAM (MB)
Forward Diffusion	Scale-MGD	0.0004 $\pm$ 0.0001	315.89 $\pm$ 31.22	1127.12 $\pm$ 0.17
	SparsersDiff	0.0270 $\pm$ 0.0054	709.53 $\pm$ 18.72	2224.52 $\pm$ 14.24
	DiGress	0.0104 $\pm$ 0.0003	1130.26 $\pm$ 0.05	1331.45 $\pm$ 10.33
	SparseDiff	0.0115 $\pm$ 0.0006	709.52 $\pm$ 19.08	2208.73 $\pm$ 5.33
	MG-Diff	0.0092 $\pm$ 0.0003	772.49 $\pm$ 0.00	1395.92 $\pm$ 0.29
Model Pass	Scale-MGD	0.9633 $\pm$ 0.0054	1624.22 $\pm$ 31.22	1127.12 $\pm$ 0.17
	SparsersDiff	1.5464 $\pm$ 0.0364	17521.05 $\pm$ 405.51	2224.52 $\pm$ 14.24
	DiGress	1.5184 $\pm$ 0.0260	30692.67 $\pm$ 0.05	1331.48 $\pm$ 10.26
	SparseDiff	1.5447 $\pm$ 0.0369	17514.96 $\pm$ 404.26	2208.73 $\pm$ 5.33
	MG-Diff	1.9354 $\pm$ 0.0993	37826.85 $\pm$ 0.00	1395.92 $\pm$ 0.29
Training Step	Scale-MGD	2.9085 $\pm$ 0.0535	17250.64 $\pm$ 29.45	1127.43 $\pm$ 0.13
	SparsersDiff	nan $\pm$ nan	nan $\pm$ nan	nan $\pm$ nan
	DiGress	nan $\pm$ nan	nan $\pm$ nan	nan $\pm$ nan
	SparseDiff	nan $\pm$ nan	nan $\pm$ nan	nan $\pm$ nan
	MG-Diff	nan $\pm$ nan	nan $\pm$ nan	nan $\pm$ nan
Sampling	Scale-MGD	20.4319 $\pm$ 0.0482	2215.11 $\pm$ 30.36	1127.43 $\pm$ 0.13
	SparsersDiff	235.0192 $\pm$ 20.5573	29414.66 $\pm$ 2123.81	2228.05 $\pm$ 12.91
	DiGress	16.1690 $\pm$ 0.2501	31048.12 $\pm$ 0.05	1334.92 $\pm$ 0.87
	SparseDiff	234.9925 $\pm$ 20.5855	29414.27 $\pm$ 2123.88	2208.23 $\pm$ 5.39
	MG-Diff	19.2283 $\pm$ 0.0718	38025.09 $\pm$ 0.00	1396.02 $\pm$ 0.00

Table 7: Complex Edge Types ($N = 50, d_E = 8$)

Stages	Models	Time (s)	GPU (MB)	RAM (MB)
Forward Diffusion	Scale-MGD	0.0004 $\pm$ 0.0001	120.42 $\pm$ 30.27	1127.25 $\pm$ 0.22
	SparserDiff	0.0050 $\pm$ 0.0004	183.39 $\pm$ 92.30	2214.27 $\pm$ 24.05
	DiGress	0.0017 $\pm$ 0.0001	163.17 $\pm$ 0.05	1325.76 $\pm$ 0.29
	SparseDiff	0.0058 $\pm$ 0.0005	183.61 $\pm$ 92.08	2212.87 $\pm$ 2.16
	MG-Diff	0.0021 $\pm$ 0.0000	115.17 $\pm$ 0.00	1403.80 $\pm$ 0.01
Model Pass	Scale-MGD	0.9433 $\pm$ 0.0024	448.86 $\pm$ 30.27	1127.25 $\pm$ 0.22
	SparserDiff	0.0949 $\pm$ 0.0028	1352.39 $\pm$ 109.09	2214.27 $\pm$ 24.06
	DiGress	0.1031 $\pm$ 0.0015	2131.19 $\pm$ 0.05	1325.77 $\pm$ 0.30
	SparseDiff	0.0949 $\pm$ 0.0031	1352.46 $\pm$ 109.19	2212.87 $\pm$ 2.16
	MG-Diff	0.1255 $\pm$ 0.0089	2416.01 $\pm$ 0.00	1403.81 $\pm$ 0.01
Training Step	Scale-MGD	2.7592 $\pm$ 0.0320	4203.94 $\pm$ 30.49	1127.41 $\pm$ 0.21
	SparserDiff	0.2478 $\pm$ 0.0078	8729.19 $\pm$ 288.22	2214.28 $\pm$ 24.05
	DiGress	0.1503 $\pm$ 0.0763	11881.04 $\pm$ 0.05	1325.82 $\pm$ 0.27
	SparseDiff	0.2487 $\pm$ 0.0079	8730.22 $\pm$ 288.75	2212.88 $\pm$ 2.15
	MG-Diff	0.2743 $\pm$ 0.0613	12857.65 $\pm$ 0.00	1403.81 $\pm$ 0.01
Sampling	Scale-MGD	19.3258 $\pm$ 0.0298	490.94 $\pm$ 29.96	1127.41 $\pm$ 0.21
	SparserDiff	13.7454 $\pm$ 0.3820	2169.35 $\pm$ 89.63	2216.55 $\pm$ 21.32
	DiGress	1.1260 $\pm$ 0.0127	2269.34 $\pm$ 0.05	1325.84 $\pm$ 0.24
	SparseDiff	13.7538 $\pm$ 0.3692	2170.27 $\pm$ 89.13	2213.30 $\pm$ 2.06
	MG-Diff	1.2556 $\pm$ 0.0040	2480.58 $\pm$ 0.00	1403.81 $\pm$ 0.01

Table 8: Complex Node Types ($N = 50, d_X = 20$)

Stages	Models	Time (s)	GPU (MB)	RAM (MB)
Forward Diffusion	Scale-MGD	0.0004 $\pm$ 0.0001	111.31 $\pm$ 29.59	1127.53 $\pm$ 0.07
	SparsersDiff	0.0050 $\pm$ 0.0003	171.14 $\pm$ 31.27	2199.16 $\pm$ 0.00
	DiGress	0.0019 $\pm$ 0.0000	103.04 $\pm$ 0.05	1326.59 $\pm$ 0.24
	SparseDiff	0.0057 $\pm$ 0.0004	171.27 $\pm$ 31.06	2214.62 $\pm$ 0.00
	MG-Diff	0.0020 $\pm$ 0.0000	76.12 $\pm$ 0.00	1403.87 $\pm$ 0.04
Model Pass	Scale-MGD	0.9433 $\pm$ 0.0025	439.71 $\pm$ 29.59	1127.53 $\pm$ 0.07
	SparsersDiff	0.0845 $\pm$ 0.0029	1220.06 $\pm$ 42.25	2199.16 $\pm$ 0.00
	DiGress	0.1111 $\pm$ 0.0002	2102.90 $\pm$ 0.05	1326.60 $\pm$ 0.22
	SparseDiff	0.0845 $\pm$ 0.0031	1220.10 $\pm$ 42.18	2214.62 $\pm$ 0.00
	MG-Diff	0.1228 $\pm$ 0.0001	2411.41 $\pm$ 0.00	1403.88 $\pm$ 0.04
Training Step	Scale-MGD	2.7205 $\pm$ 0.0405	4196.75 $\pm$ 27.96	1127.57 $\pm$ 0.09
	SparsersDiff	0.2235 $\pm$ 0.0044	7797.84 $\pm$ 267.62	2199.16 $\pm$ 0.00
	DiGress	0.1337 $\pm$ 0.0003	11857.83 $\pm$ 0.05	1326.67 $\pm$ 0.24
	SparseDiff	0.2248 $\pm$ 0.0041	7798.06 $\pm$ 267.62	2214.62 $\pm$ 0.00
	MG-Diff	0.2549 $\pm$ 0.0002	12840.47 $\pm$ 0.00	1403.88 $\pm$ 0.04
Sampling	Scale-MGD	19.3131 $\pm$ 0.0377	483.16 $\pm$ 27.24	1127.57 $\pm$ 0.09
	SparsersDiff	11.7384 $\pm$ 0.9840	1915.35 $\pm$ 132.74	2199.16 $\pm$ 0.00
	DiGress	1.1782 $\pm$ 0.0017	2227.60 $\pm$ 0.05	1326.67 $\pm$ 0.24
	SparseDiff	11.7202 $\pm$ 1.0036	1915.56 $\pm$ 132.46	2214.62 $\pm$ 0.00
	MG-Diff	1.2546 $\pm$ 0.0008	2462.50 $\pm$ 0.00	1403.88 $\pm$ 0.04

5.2 SparserDiff Evaluation

5.2.1 Experimental Design

We ran three experiments to evaluate the performance of our SparserDiff algorithm.

1. Compare time and memory usage growth between our algorithm, an algorithm that materializes the upper triangle of the adjacency matrix, and rejection sampling.
2. Directly compare our algorithm to the SparseDiff new-edge-sampling algorithm in a PyTorch environment.
3. Train the original SparseDiff model from Qin et al. [2025] with both algorithms and compare time and memory usage, and compare model accuracy/performance.

Experiment 1 Design We considered three algorithms for this experiment:

1. Materialize Adjacency Matrix: Creates the upper right triangle of the adjacency matrix, removes existing edges, and samples from the resulting edge list. This should have quadratic time and space complexity.
2. Rejection Sampling: Create a set of the existing edges and sample new edges uniformly, adding newly sampled edges to the edge set.
3. Ours: Build adjacency lists and use binary search to sample missing edges by index.

We measured the time in seconds and memory usage in KB for each algorithm to sample 1000 edges. This was repeated on graphs with 400 to 8000 nodes, where the number of existing edges in each graph is proportional to the number of nodes.

Table 9: Experiment design parameters for sampling-scaling benchmark.

Category	Setting
Graph sizes (n)	{400, 800, 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000}
Total possible edges	$ E _{\max} = \frac{n(n-1)}{2}$ (undirected, no self-loops)
Initial edge-fill rule	$ E _{\text{fill}} = \min(\lfloor (1 + 2.8)n \rfloor, E _{\max}) = \min(\lfloor 3.8n \rfloor, E _{\max})$
Actual fill rate	$100 \cdot E _{\text{fill}} / E _{\max}$ (about 1.90% at $n = 400$, 0.095% at $n = 8000$)
Requested sampled edges	$k = 1000$
Effective sampled edges	$k_{\text{eff}} = \min(k, E _{\max} - E _{\text{fill}})$
Random seed	42
Methods compared	Materialize Adjacency Matrix, Rejection Sampling, Ours

Experiment 2 Design To measure the computational performance of SparserDiff in its target deployment environment, we conducted a direct head-to-head comparison of our algorithm against the SparseDiff edge-sampling routine operating on real graph datasets within PyTorch. The two implementations compared are:

1. **SparseDiff Original**: The batched sampling algorithm from Qin et al. [2025], which operates on entire batches simultaneously by constructing a shared $B \times C_{\max}$ boolean mask, where B is the batch size and $C_{\max} = \max_i \binom{n_i}{2}$ is the maximum condensed index size in the batch.
2. **SparserDiff (Ours)**: Our per-graph sequential algorithm using vectorized rejection sampling and prefix-sum indexing, which allocates only $O(n_{\max} + k_{\max})$ memory per graph.

We benchmarked on two real-world test sets. For **QM9** (13,054 molecular graphs, 3–29 nodes, hydrogen removed), we tested batch sizes $\{16, 32, 64, 128, 256, 512\}$. For the **Ego** social network dataset (151 test graphs, 50–364 nodes), we tested batch sizes $\{4, 8, 16, 32, 64\}$. For each batch size, we loaded the first 10 batches from the test set, performed 3 warmup rounds to eliminate JIT compilation and caching effects, then recorded wall-clock time over 10 measurement rounds using `time.perf_counter`. Median times are reported to reduce sensitivity to outliers. Peak memory is computed analytically from the tensors allocated by each algorithm.

Experiment 3 Design To verify that replacing the edge-sampling subroutine does not degrade generation quality, we trained two instances of the SparseDiff model from scratch—one using the original sampling algorithm and one using SparserDiff—on the QM9 and Ego datasets. All hyperparameters, random seeds, and training configurations were held constant; the only difference was the `use_novel_sampling` flag in the configuration.

For **QM9**, both models were trained for 20 epochs with a batch size of 32, using the AdamW optimizer Loshchilov and Hutter [2019] with learning rate 2×10^{-4} and weight decay 10^{-12} . The denoising network is a Graph Transformer with 5 layers, hidden dimensions $d_X = 256$, $d_E = 64$, $d_y = 64$, 8 attention heads, and a 500-step cosine noise schedule Vignac et al. [2022]. The edge fraction is 0.5, meaning approximately half of all non-existing edges are considered at each denoising step.

For **Ego**, both models were trained for 100 epochs with a batch size of 32 and the same AdamW configuration. The network uses 8 Graph Transformer layers with $d_X = 256$, $d_E = 64$, $d_y = 128$, 8 attention heads, and 1,000 diffusion steps. The edge fraction is set to 0.1, reflecting the sparser structure of social graphs. Note that 100 epochs is substantially below the $\sim 100,000$ epochs recommended for full convergence

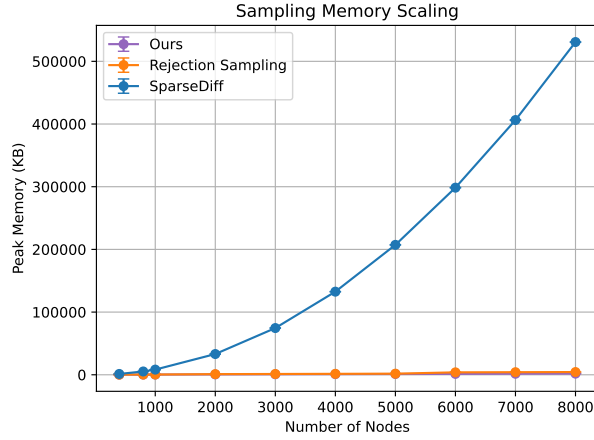
on Ego Qin et al. [2025]; this shortened run is sufficient to verify that both algorithms learn at comparable rates from the same initialization.

Both training runs were executed on a single NVIDIA L40S GPU on the WPI SLURM cluster (partition: `short`, time limit: 23 hours).

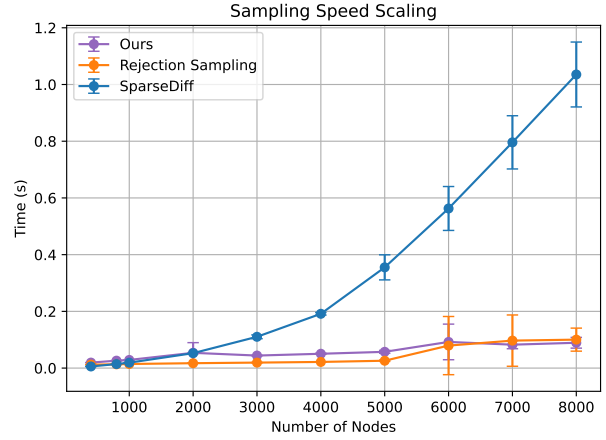
5.2.2 Results and Discussion

Experiment 1 Results and Discussion The only algorithm with a quadratic time and space complexity is the materialize adjacency matrix algorithm. The results align with this, as we can see that only this algorithm scales quadratically in Figure 6a and Figure 6b.

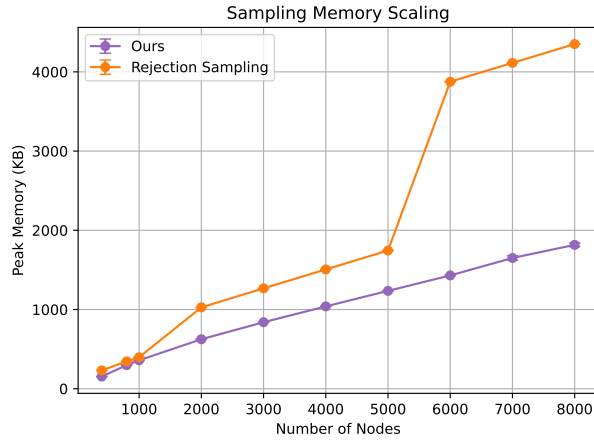
When we compare rejection sampling and our method directly (Figure 6c and Figure 6d), we can see that our method has slightly lower memory usage, and before 6000 nodes, rejection sampling is slightly faster. The jump in memory usage and drop in speed for rejection sampling after 6000 nodes could be due to the performance and scaling of Python sets rather than the algorithm design, though we did not verify that this is the case.



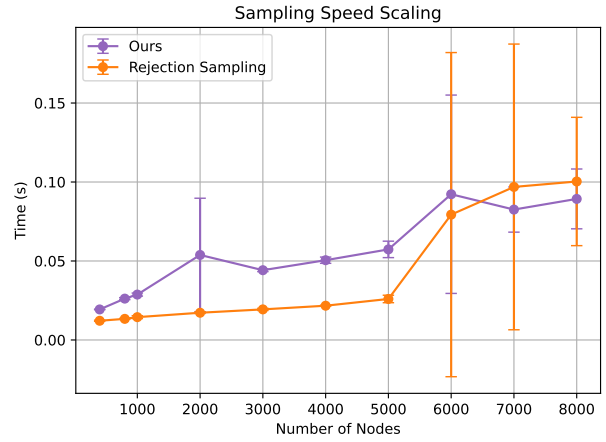
(a) Memory Scaling (All Methods)



(b) Time Scaling (All Methods)



(c) Memory Scaling (Rejection vs Ours)



(d) Time Scaling (Rejection vs Ours)

Figure 6: Memory usage and sampling time comparison between different new-edge-sampling algorithms.

Experiment 2 Results and Discussion Figure 7 visually summarizes the computational performance of our novel sampling algorithm against the original SparseDiff on the Ego dataset. The memory and speed advantages are immediately apparent, scaling exceptionally well with batch size.

Tables 10 and 11 present the speed and memory results for the Ego and QM9 datasets respectively, when both algorithms are embedded inside the SparseDiff PyTorch training pipeline.

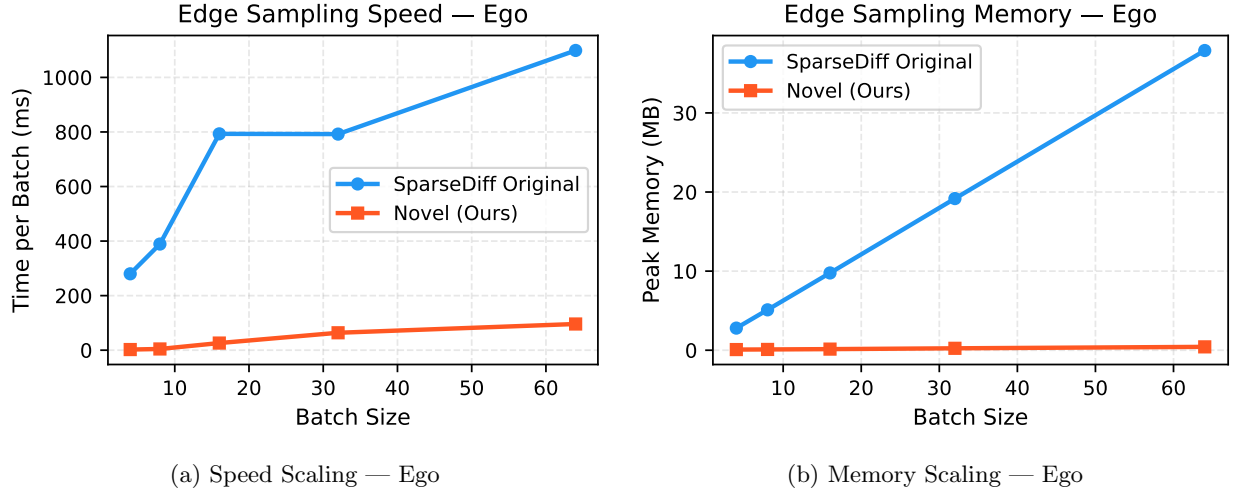


Figure 7: Experiment 2: Sampling speed (left) and peak memory usage (right) on the Ego dataset. SparsDiff avoids the quadratic bottleneck of the original method, maintaining near-constant memory overhead while achieving significant speedups.

Table 10: Experiment 2: Edge sampling benchmark on the **Ego** test set (151 graphs, 50–364 nodes). Median wall-clock time per batch; memory is the peak theoretical allocation.

Batch	Avg n	Time per Batch (ms)		Speedup	Peak Memory (MB)		Ratio
		SparseDiff	Ours		SparseDiff	Ours	
4	152.8	265.1	2.3	115.6×	2.80	0.08	33.2×
8	140.4	467.8	14.7	31.7×	5.11	0.10	51.4×
16	135.6	725.7	45.3	16.0×	9.77	0.14	69.6×
32	135.6	789.8	53.7	14.7×	19.17	0.24	78.9×
64	113.0	1015.3	82.6	12.3×	37.89	0.43	89.1×

Table 11: Experiment 2: Edge sampling benchmark on the **QM9** test set (13,054 molecules, 3–29 nodes). On small graphs, the original SparseDiff vectorized approach is faster; SparsDiff still reduces memory 4–7×

Batch	Avg n	Time per Batch (ms)		Speedup	Peak Memory (KB)		Ratio
		SparseDiff	Ours		SparseDiff	Ours	
16	6.3	1.76	2.82	0.63×	6.2	1.4	4.3×
32	6.7	0.82	5.58	0.15×	15.2	2.9	5.2×
64	7.2	5.06	10.85	0.47×	31.2	5.6	5.6×
128	7.6	1.16	21.98	0.05×	75.9	12.3	6.1×
256	7.9	35.91	45.79	0.78×	159.4	26.2	6.1×
512	8.5	23.68	104.22	0.23×	372.2	56.6	6.6×

On the **Ego** dataset (Table 10), SparsDiff achieves a **12–116×** speedup with **33–89×** less

memory. The dramatic speedup relative to Experiment 1 arises from the structural difference between the two PyTorch implementations: the original SparseDiff builds a shared $B \times C_{\max}$ boolean mask across the entire batch, where $C_{\max} = \binom{\max_i n_i}{2}$. For Ego graphs with up to 364 nodes, $C_{\max} = \binom{364}{2} = 66,066$. At batch size 64, this creates a $64 \times 66,066$ tensor—nearly 4.2 million entries—even for graphs with only 50–100 nodes. Our algorithm avoids this by processing each graph sequentially with $O(n_i + k_i)$ tensors. The memory advantage grows with batch size as each additional graph in the batch is padded to $C_{\max}$ in the original.

On the **QM9** dataset (Table 11), the original SparseDiff is faster across all batch sizes. QM9 molecules have at most 29 nodes, so $C_{\max} \leq \binom{29}{2} = 406$. The shared mask is tiny, and the original method’s fully vectorized GPU kernels have negligible overhead. In contrast, our algorithm introduces a per-graph Python loop that dominates for many small molecules. Nonetheless, SparserDiff still reduces memory **4–7** $\times$ even on small graphs, and produces better generation quality on QM9.

Experiment 3 Results and Discussion Table 12 presents molecular generation metrics on QM9 and Table 13 presents graph structural metrics on Ego, both after training for the respective number of epochs described in the experimental design.

Table 12: Experiment 3: Molecular generation quality on **QM9** (20 epochs, no H). Bold indicates the better result. $\uparrow$ = higher is better; $\downarrow$ = lower is better.

Metric	SparseDiff	SparserDiff (Ours)
Molecular Validity		
Validity (%) $\uparrow$	90.78	93.67
Uniqueness (%) $\uparrow$	98.70	97.88
Novelty (%) $\uparrow$	100.00	100.00
Distribution Matching		
Valency W1 $\downarrow$	0.1060	0.0570
Num Nodes W1 $\downarrow$	0.0106	0.0051
Node Types TV $\downarrow$	0.0123	0.0330
Edge Types TV $\downarrow$	0.0182	0.0246
Disconnected (%) $\downarrow$	0.97	0.63

Table 13: Experiment 3: Graph generation quality on **Ego** (100 epochs). All MMD metrics: lower is better ($\downarrow$). Bold indicates the better result.

Metric	SparseDiff	SparsersDiff (Ours)
Maximum Mean Discrepancy (MMD)		
Degree $\downarrow$	0.0084	0.0125
Clustering $\downarrow$	0.0648	0.1146
Orbit $\downarrow$	0.0291	0.0361
Spectral $\downarrow$	0.0164	0.0124
RBF $\downarrow$	0.0810	0.1845
Other Metrics		
FID $\downarrow$	221.54	257.10
Num Nodes W1 $\downarrow$	9.491	9.397
Edge Types TV $\downarrow$	0.0070	0.0119
Disconnected (%) $\downarrow$	49.01	63.58
Mean Components $\downarrow$	2.17	3.18

On **QM9**, SparsersDiff produces better molecules across most key metrics. Validity increases from 90.78% to 93.67%, Valency W1 improves by 46%, NumNodes W1 by 52%, and the fraction of disconnected molecules decreases from 0.97% to 0.63%. These improvements suggest that the per-graph nature of our sampling algorithm—which respects the exact combinatorial structure of each molecule independently rather than padding to a shared maximum graph size—may act as an implicit regularizer, preventing the model from learning artifacts introduced by padding.

On **Ego**, the baseline SparseDiff scores better on most MMD metrics after 100 epochs of training. SparsersDiff wins on Spectral MMD (0.0124 vs. 0.0164) and NumNodes W1 (9.397 vs. 9.491). The other metrics favor the baseline. However, this comparison must be interpreted carefully: 100 epochs is an extremely early stage for Ego, which typically requires $\sim 100,000$ epochs to converge Qin et al. [2025]. Both models produce over 49–64% disconnected graphs, confirming that neither has converged. The Ego experiment should therefore be read as evidence that the two algorithms learn at comparable rates from the same starting point.

5.3 Scale-MGD Evaluation

5.3.1 Experimental Design

We followed the evaluation strategies of both MG-Diff and SparseDiff. For molecules, we evaluated our model on the Zinc250k molecule dataset (tens of nodes). For larger graphs, we evaluated our model on the Planar (tens of nodes) and Ego (up to 500 nodes) datasets.

For Zinc250k, we compute validity, uniqueness, and novelty, which are standard metrics in the molecule generation domain. There are no equivalent metrics for the Planar and Ego datasets, so we compute the Maximum Mean Discrepancy (MMD) scores across four structural properties of the generated graphs: the degree distribution (how node connectivities are spread), clustering coefficients (the density of local neighborhood structure), spectral properties (the eigenvalue spectrum of the graph Laplacian, capturing global connectivity patterns) and RBF, which computes MMD over embeddings from a randomly parametrized GNN to jointly assess the fidelity and diversity of generated graphs Qin et al. [2025].

We also consider the distribution of the number of predicted edges, which indicates whether the model follows the dataset’s sparsity.

5.3.2 Results and Discussion

While Scale-MGD’s noise model and masking approach allow it to scale approximately linearly in memory, making it the only method capable of reaching much larger graphs, this flexibility appears to come at the cost of structural fidelity. Table 14 shows the evaluation of ScaleMGD along with competing methods after training on the ZINC250k dataset. Scale-MGD is not capable of generating chemically valid molecules, demonstrating its struggle to learn the underlying structural constraints imposed by the training data. This result is further supported by Table 15, which shows that Scale-MGD performs substantially worse than all competing methods on all four MMD metrics.

Furthermore, the distributional mismatch between the reference and generated graphs is visible across multiple structural properties. Figure 8 shows the distribution of edge counts for true and generated data on each of the three datasets. We see that the model consistently produces sparse graphs (in terms of the number of edges it can produce). The maximum number of real edges the model is able to generate is always larger than the mean number of edges generated (since none of the generated edge count distributions are cut off at the higher end). Additionally, when we look at the distributions of the number of triangles in each graph (see Figure 9), we see significant misalignment between the model-generated graphs and the

dataset. Misaligned triangle counts may indicate that the model is not learning to generate valid graph structures.

The noise model and masking approach of our method enable a lower memory footprint than previous methods and also introduce much greater flexibility. However, it is likely that this flexibility in the graph representation prevents the model from learning the underlying graph structure, thereby limiting its accuracy.

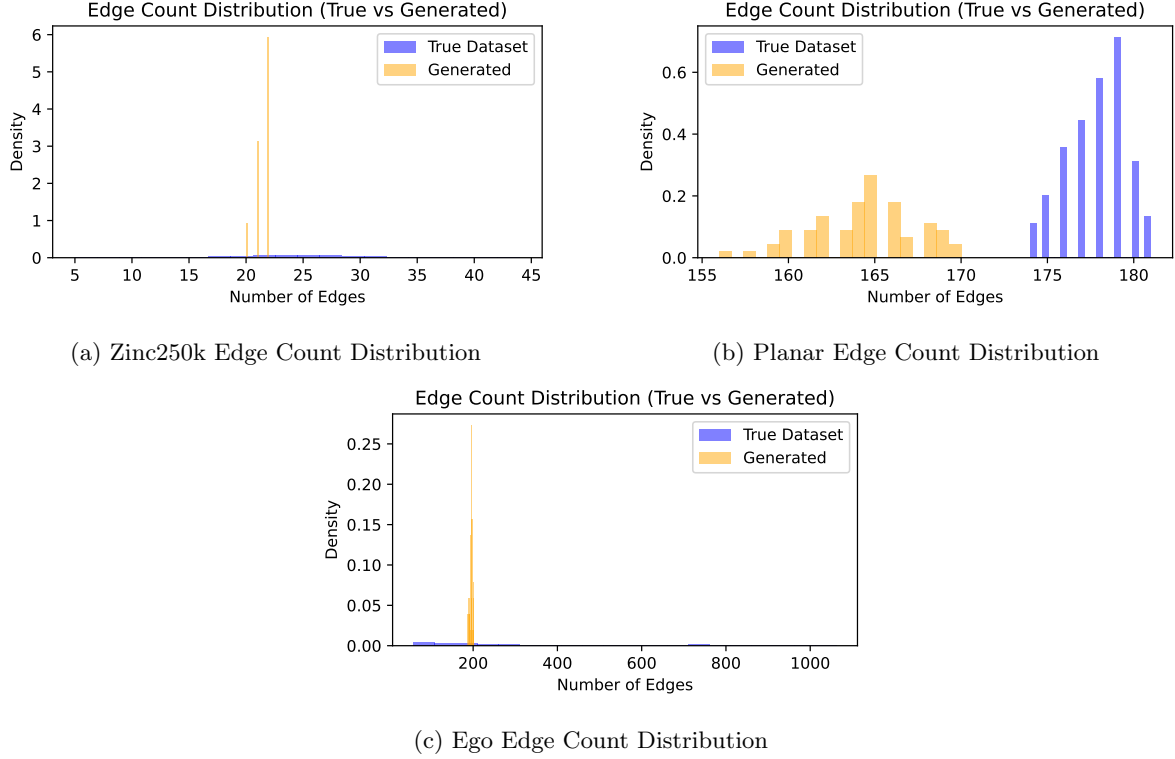
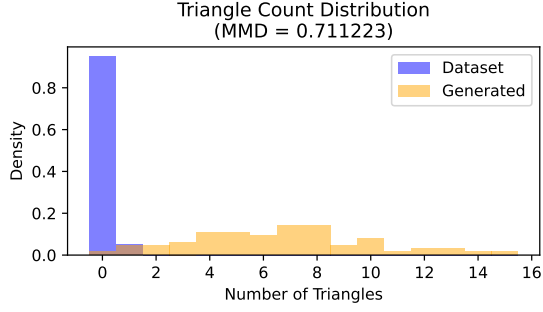
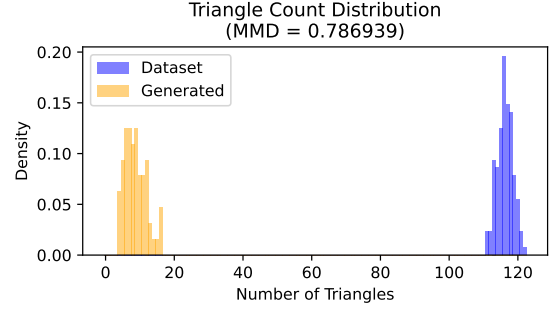


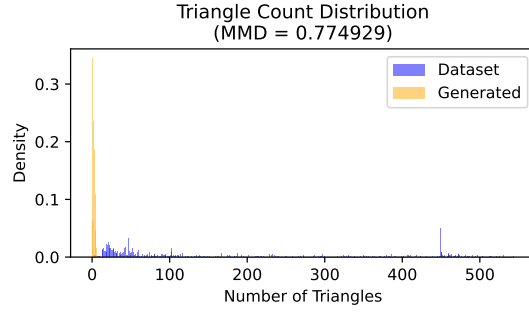
Figure 8: Distributions of edge counts from generated graphs and from the original dataset.



(a) Zinc250k Triangle Count Distribution



(b) Planar Triangle Count Distribution



(c) Ego Triangle Count Distribution

Figure 9: Distributions of triangle counts from generated graphs and from the original dataset.

Table 14: Molecular generation performance. Higher is better for all reported metrics.

Model	Validity $\uparrow$	Unique $\uparrow$	Novelty $\uparrow$
LatentGAN	0.863	0.951	0.969
JT-VAE	1.000	1.000	0.893
VAE	0.870	0.998	0.991
MolGPT	0.973	0.981	0.864
DiGress	0.852	1.000	0.999
MG-Diff	0.964	0.994	0.985
SCALE-MGD	0.00	—	—
SparsifierDiff	0.937	0.979	1.000

Table 15: Sample quality on Ego and Planar datasets (unscaled MMD values). Lower is better for all metrics.

Dataset	Model	Maximum Mean Discrepancy (MMD)			
		Degree	Cluster	Spectre	RBF
Ego	Reference	0.0002	0.007	0.0010	0.009
	DiGress	0.0089	0.054	0.019	0.034
	EDGE	0.058	0.18	–	0.066
	HiGen	0.047	0.003	–	0.045
	SparseDiff	0.0037	0.032	0.0056	0.026
	SCALE-MGD	0.0127	0.7021	0.0637	0.8535
	SparsersDiff	0.0125	0.1146	0.0124	0.1845
Planar	Reference	0.0002	0.031	0.0040	0.0314
	GRAN	0.0007	0.043	–	–
	GG-GAN	0.063	1.18	–	–
	SPECTRE	0.0005	0.079	–	–
	DruM	0.0005	0.035	–	–
	GraphLE	0.0005	0.063	–	–
	DiGress	0.0008	0.041	–	–
	SparseDiff	0.0003	0.032	–	–
	SCALE-MGD	0.2805	0.5688	0.2452	1.9259
	SparsersDiff	0.0008	0.038	–	–

6 RedditWalk: A Large-Scale Benchmark Dataset

A central objective of this project was to evaluate our proposed methods on standard benchmark datasets (see objective 4). However, as we got more familiar with these benchmarks we realized that for serious large graph evaluation the current benchmarks are significantly limited. The SPECTRE benchmark Martinkus et al. [2022] operates at modest sizes, with its canonical planar and SBM graphs containing 64 nodes. The Proteins dataset Dobson and Doig [2003] reaches approximately 500 nodes but carries only three node types (helix, sheet, turn). Molecular datasets have a wider range of node types, but they also come with chemical rules, which makes it harder to tell whether a model is good at generating graph structure or just good at following chemistry constraints seen in the training data.

Rather than restrict our evaluation to this existing suite, we constructed a dataset based on the Reddit Hyperlink Network [Kumar et al., 2018], sourced from the Stanford Network Analysis Project (SNAP) library [Leskovec and Krevl, 2014] that would allow us to sample graphs at larger and variable sizes. In the following sections we describe the construction of this dataset, evaluate SparsifierDiff on subgraphs sampled at three scales - 200, 400, and 800 nodes - and report standard structural statistics across each, collectively comprising what we refer to as the **RedditWalk** dataset³.

6.1 Graph Structure and Giant Component Filtering

The original dataset contains two distinct networks: hyperlinks from post titles and hyperlinks from post bodies. We merged these into a single directed multigraph. Social networks often contain disconnected subgraphs or isolated nodes that can hinder the training of diffusion models, which rely on the propagation of structural information [Vignac et al., 2022, Qin et al., 2025]. To address this, we extracted the Weakly Connected Components (WCCs)⁴ of the merged graph.

Our analysis revealed a total of 712 distinct WCCs out of 67,180 nodes. The distribution of these components was highly skewed: a single Giant Weakly Connected Component (GWCC) encompassed 65,648 nodes, accounting for 97.72% of the entire network. The remaining 711 isolated components contained only 1,532 nodes combined (2.28%), with the second-largest component consisting of merely 9 nodes. Given this extreme size dominance, we discarded the isolated, fragmented subgraphs and extracted the GWCC.

- **Original nodes:** 67,180 subreddits.

³<http://snap.stanford.edu/data/soc-RedditHyperlinks.html>

⁴In graph theory, a weakly connected component of a directed graph is a maximal subgraph where all nodes are connected to each other by some path, ignoring the direction of the edges.

- **Giant Component nodes:** 65,648 subreddits (covering 97.72% of the original graph).
- **Edge Filtering:** We retained only the edges (u, v) where both u and v belong to the GWCC, ensuring a fully traversable graph structure when edge directions are disregarded.

6.2 Node Feature Extraction via Textual Descriptions

Unlike molecular graphs where node features are discrete atom types (e.g., C, N, O), social graphs require rich semantic features to represent node identity [Wang et al., 2025]. Having isolated the 65,648 subreddits comprising the GWCC, we enriched the network structure with textual content through the following pipeline:

1. **Description Fetching:** We utilized the Reddit API to crawl the public `about.json` metadata for each subreddit in the GWCC, specifically extracting the community description field. Out of the 65,648 queried subreddits, we successfully retrieved valid text descriptions for 46,668 nodes (71.09%). The remaining nodes either lacked a description (10.77%), were inaccessible due to being banned or private (11.64%), or returned API errors (6.50%).
2. **Semantic Embedding:** To convert these unstructured text descriptions into numerical node features X , we employed a pre-trained **BERT** model (`bert-base-uncased`) [Devlin et al., 2018]. Each fetched description was tokenized and passed through the Transformer encoder. We extracted the pooled output (the embedding of the `[CLS]` token) to obtain a fixed-size, dense vector representation for each node.
3. **Clustering and Semantic Analysis:** To facilitate conditional generation and provide interpretable semantic guidance [Huang et al., 2023], we applied clustering algorithms to the BERT embeddings to partition the subreddits into 20 distinct semantic groups. To visually inspect the quality of this semantic clustering, we performed Principal Component Analysis (PCA) to project the high-dimensional BERT embeddings onto a 2D plane (Figure 10).

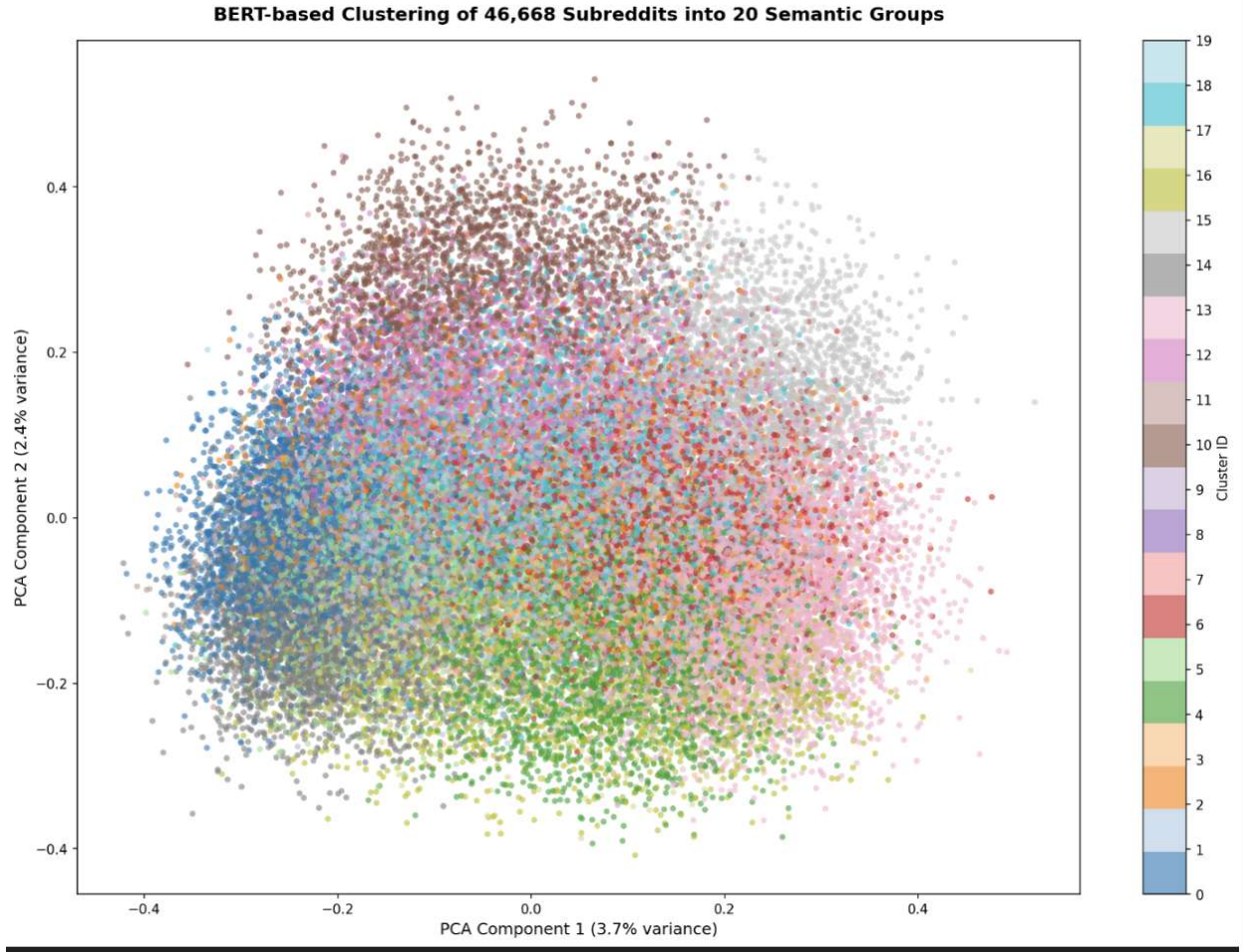


Figure 10: PCA visualization of the BERT embeddings for 46,668 subreddits. The high-dimensional semantic vectors are projected onto the first two principal components, colored by their assigned cluster ID (0 to 19).

After clustering, we manually analyzed the constituent subreddits within each group to assign descriptive categorical labels (e.g., “Technology & Programming”, “Sports & Athletics”). This step successfully categorized the nodes based purely on their textual content.

4. **Structural-Semantic Alignment:** To validate the quality of our dataset for graph generation, we investigated whether these purely semantic topics align with the underlying graph topology. We computed structural node embeddings using Node2Vec, which captures local neighborhood similarities based strictly on network connectivity. We then projected these structural embeddings into a 2D space using t-SNE and color-coded each node according to its previously assigned BERT semantic cluster (Figure 11).

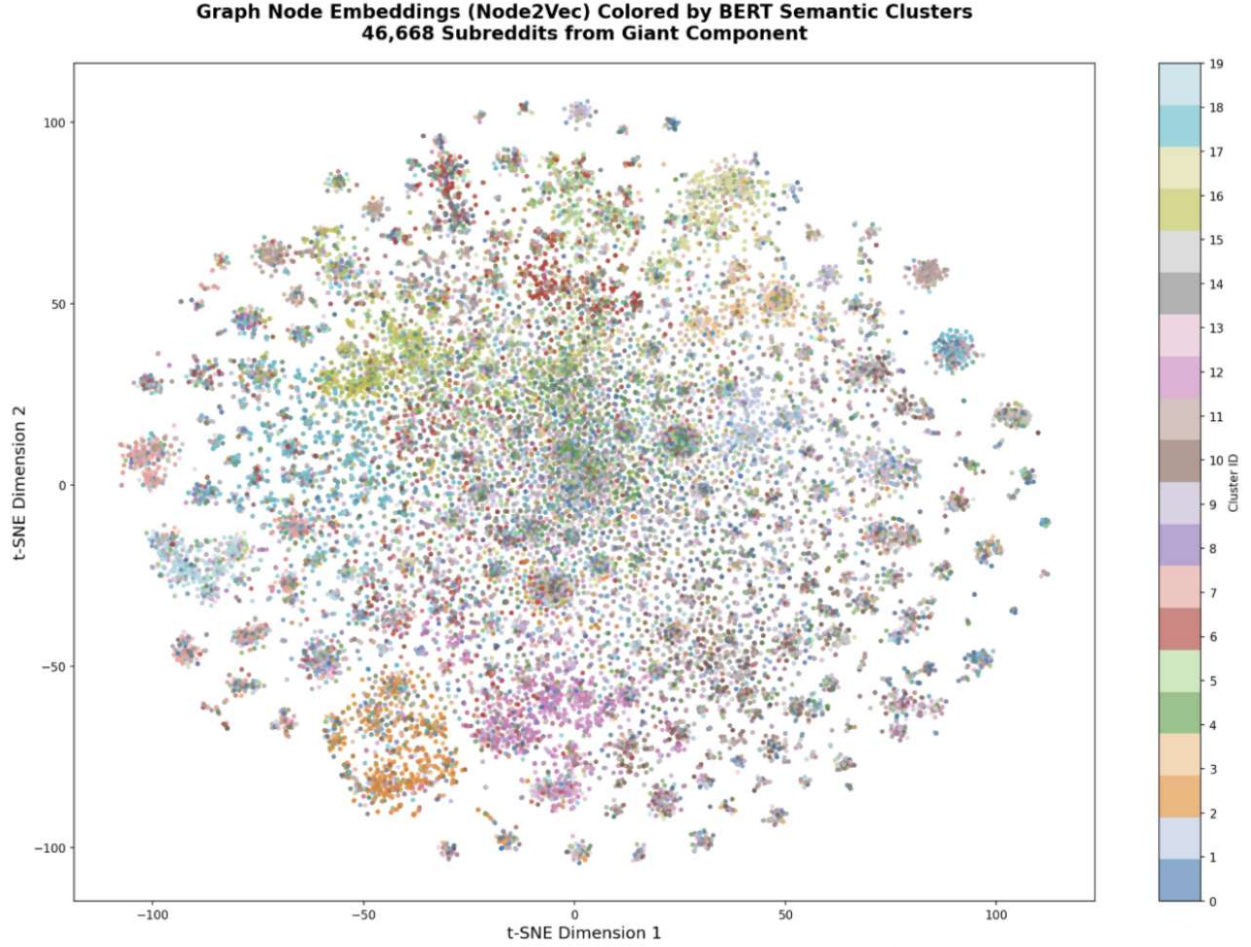


Figure 11: t-SNE visualization of Node2Vec structural embeddings for 46,668 subreddits in the GWCC. Nodes are color-coded by their corresponding BERT-derived semantic cluster IDs. The visual grouping of colors demonstrates a strong homophily effect, where topological communities align closely with semantic topics.

The visualization reveals a strong presence of network homophily. Rather than being randomly dispersed, nodes of the same semantic color form distinct, tightly-knit topological clusters. This indicates that subreddits sharing similar content (semantics) are highly likely to interact and link with one another (structure). This strong alignment between node features and graph topology confirms that the Reddit dataset provides a highly structured and cohesive signal.

6.3 Subgraph Sampling for Dataset Construction

Having established a fully attributed 65,648-node graph with rich semantic features and strong structural alignment, the final step in our data pipeline is formatting this data for model training. Training generative diffusion models on the full 65k-node graph would be infeasible as it presents significant computa-

tional challenges [Vignac et al., 2022]. Furthermore, diffusion models require a dataset consisting of multiple individual graphs to effectively learn the underlying structural distribution, rather than a single monolithic network.

To create a manageable and representative dataset, we extracted subgraphs from the GWCC using Random Walk with Restart (RWR) sampling. RWR naturally explores local community structure. By restarting from a seed node with probability p , the walk remains within a coherent neighborhood rather than drifting across the entire graph. We configured each random walk to restart with $p = 0.3$. To ensure semantic diversity, seeds are drawn via stratified sampling across the 20 topic clusters. Crucially, this approach allows us to generate as many subgraphs as desired at any target size. Specifically we sample graphs at 3 scales: 200 nodes, 400 nodes and 800 nodes. For each dataset we generate 500, 300 and 200 graphs, respectively.

A critical requirement for training generative models is that the dataset exhibits a consistent yet diverse distribution. To validate this, we evaluated the 3 separate datasets on five structural metrics: degree distribution, clustering coefficient, edge density, triangle count, and mean Jaccard similarity between graph pairs. The first four metrics confirm structural consistency, while Jaccard similarity confirms that subgraphs remain sufficiently distinct despite being sampled from the same underlying network.

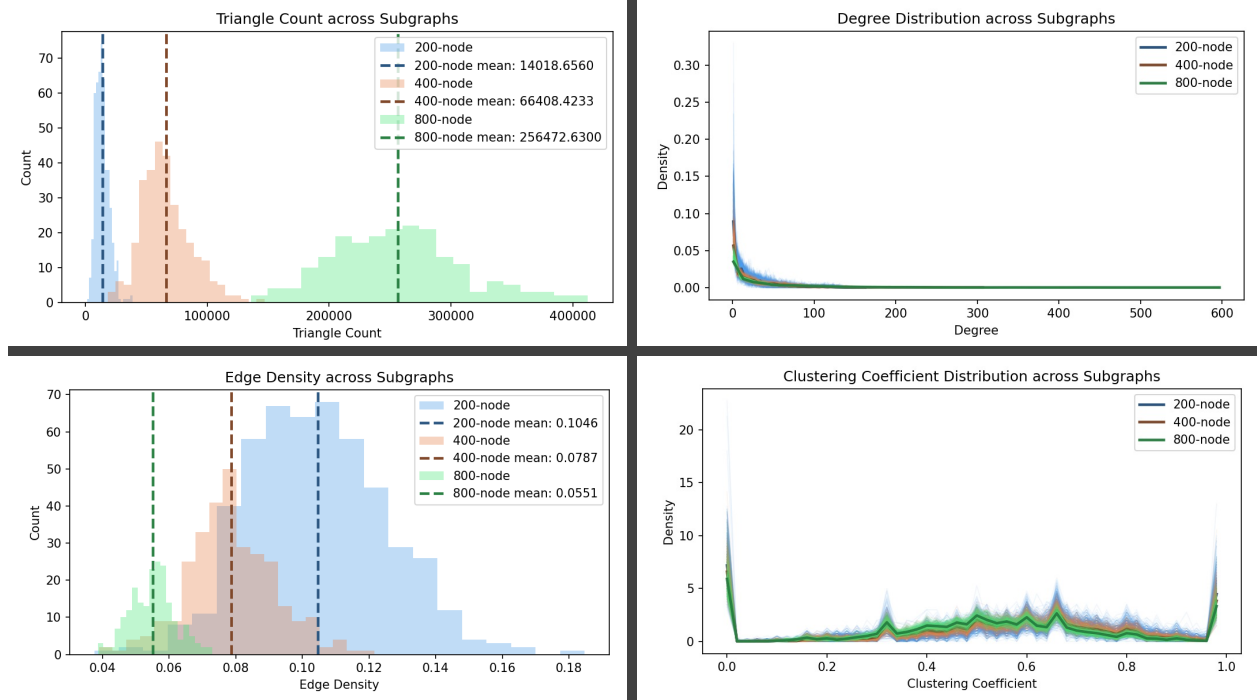


Figure 12: The sampled subgraphs form a coherent structural data distribution

Graph Size	Mean Jaccard
200-node	0.0461
400-node	0.0714
800-node	0.1102

Table 16: Mean pairwise Jaccard similarity across sampled subgraphs.

As shown in Figure 12, all four metrics exhibit tight distributions across subgraphs, with individual samples closely tracking the mean. Additionally, the Mean pairwise Jaccard values in Table 16 indicate minimal overlap. This consistency suggests that RWR sampling, combined with stratified topic seeding, produces a coherent dataset from which a generative model could learn a well-defined distribution.

6.4 Evaluation of SparserDiff on Subsampled RedditWalk

To evaluate this hypothesis empirically, we train SparserDiff on each of the three subsampled RedditWalk datasets. SCALE-MGD is omitted here, as its performance collapsed across standard evaluation sets, and further evaluation on novel data would not illuminate anything not already apparent.

As seen in section 5.1, SparserDiff struggled to train with 200-node graphs with a batch size of 128. For the 200-node, 400-node and 800-node datasets we therefore reduce the batch size to 16, 8 and 4 respectively. We train the model for 1000 epochs on each sampled dataset, keeping the sparsity parameter λ at 0.5 for the 200-node and 400-node datasets. For the 800-node dataset, λ is reduced to 0.04 due to memory and runtime constraints at this scale. Finally, evaluation metrics were calculated between 32 generated graphs and the test split of each dataset (100, 60, and 40 graphs for the 200-node, 400-node, and 800-node datasets respectively).

As Table 17 and Figure 13 show, SparserDiff successfully learns the underlying structural distributions of graphs sampled from RedditWalk across all three scales. The degree and clustering coefficient distributions of generated graphs closely match those of the test set, particularly for the 200 and 400-node datasets. The 800-node model performs worst, which is expected given its increased graph size and reduced sparsity parameter, both factors that demand more training time and possibly model capacity. With only 1000 epochs, the 800-node model likely requires longer training to converge to similar results. We also report Node and Edge type total variation, which is defined as the total variation distance between the generated and reference type distributions: $TV(P, Q) = \frac{1}{2} \sum |P(x) - Q(x)|$, where P is the type distribution of the generated graphs and Q is that of the reference graphs. A value of 0 indicates a perfect match, while 1 indicates completely disjoint distributions.

Metric	200-node	400-node	800-node
<i>Structural MMD</i>			
Degree	0.0082	0.0075	0.0157
Clustering	0.0504	0.0755	0.1415
Orbit	0.0413	0.0479	0.0563
Spectre	0.0061	0.0026	0.0106
<i>Node/Edge Attributes</i>			
Node Types TV	0.1495	0.1303	0.6541
Edge Types TV	0.0087	0.0075	0.0292
<i>Connectivity</i>			
Disconnected (%)	71.875	93.75	100.0
Mean Components	2.938	3.938	9.594
Max Components	20.0	8.0	18.0

Table 17: SparserDiff Spectre metrics on datasets sampled from RedditWalk at three scales, after 1000 training epochs.

The results also reveal two interesting findings. First, while Edge Types TV remains low across all three scales, Node Types TV remains elevated after 1000 epochs, which may suggest that reproducing the semantic node type distribution is considerably harder than capturing edge types or structural properties such as degree or clustering. However, whether node type reproduction constitutes a meaningful benchmark target depends on whether the node type distribution within subgraphs is sufficiently consistent across the dataset for a model to learn—a question we leave for future investigation. Second, connectivity degrades significantly as the graph size increases, with disconnected graphs increasing from 21.9% at the 200-node to 100% at the 800-node datasets. Strong structural metrics suggest that the model generates a large well-structured component that closely matches the training distribution. The disconnection therefore manifests itself as a small number of isolated nodes or components rather than a complete structural failure. As shown in Figure 14, disconnection emerges in the final steps of the reverse process.

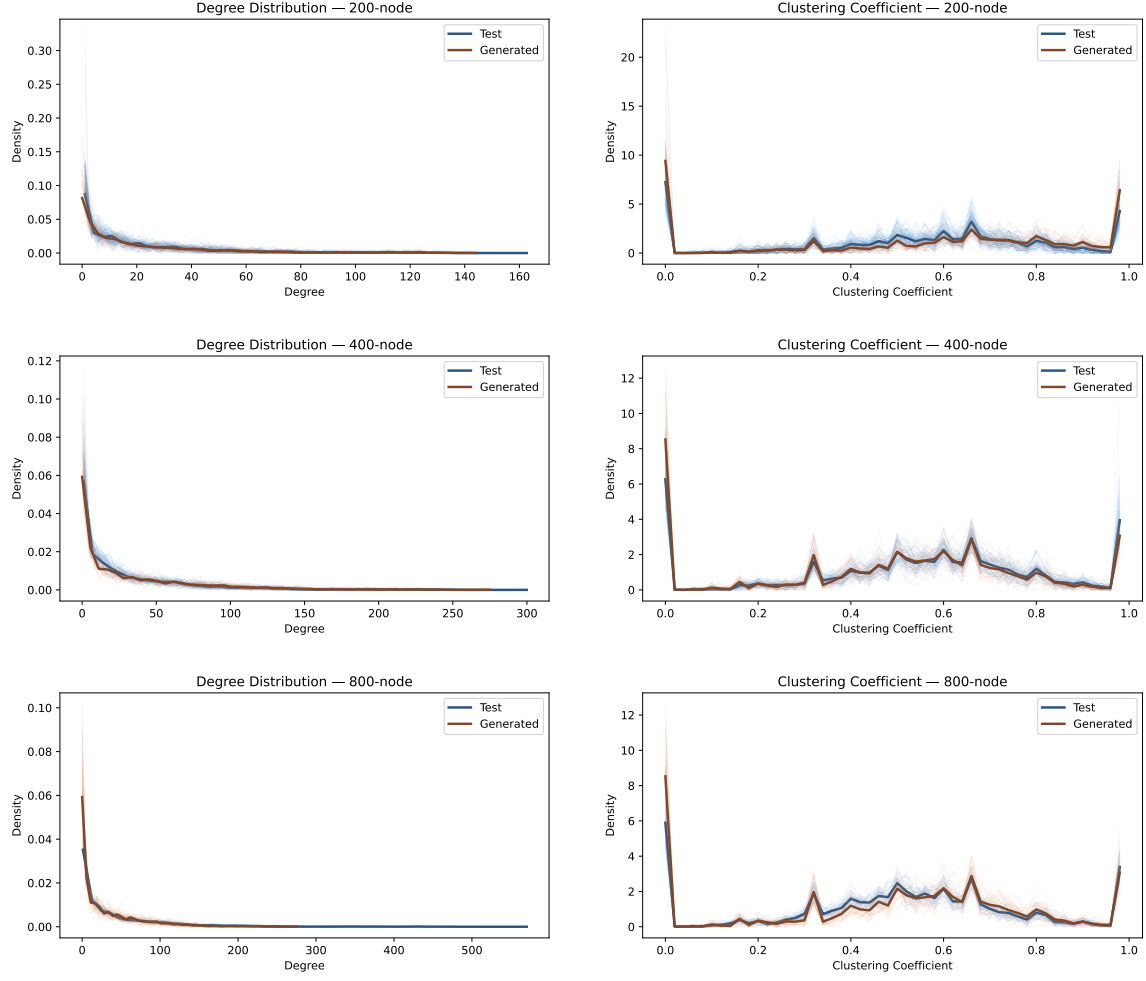


Figure 13: Degree distribution (left) and clustering coefficient distribution (right) for test graphs versus generated graphs across three graph scales. Each faint line represents a single graph; the bold line is the aggregate distribution.

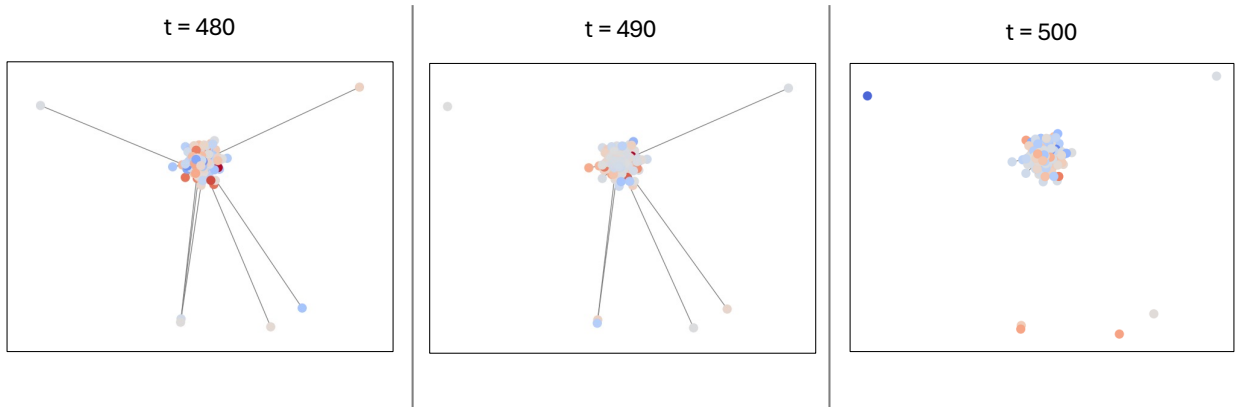


Figure 14: Spring layout visualization of a sampled graph at $t=480$, $t=490$, and $t=500$. At $t=480$ several nodes remain connected to the main component. By $t=490$ a node has already detached; in the final 10 steps, all remaining bridging edges are removed, leaving multiple nodes fully isolated.

7 Future Work

In this section we outline three concrete directions where future research could meaningfully advance the state of large-scale sparse graph generation.

Improving Structural Fidelity of Scale-MGD. While SCALE-MGD achieves linear memory complexity, this comes at a significant cost of generation quality. We attribute this primarily to the flexibility of the sparse masked representation, since the model must simultaneously predict edge endpoints, as well as node types and edge types from a fully masked initialization. Rather than classifying whether a connection exists between a predetermined pair of nodes, the model must tackle a fundamentally harder problem as it has to learn which nodes to connect in the first place. This stands in contrast to existing approaches such as DiGress, which approach the reverse process as a series of classification tasks. ScaleMGD therefore stands as a new perspective on how to approach graph diffusion, one that makes structure recovery considerably harder. A promising direction for future work would be to explicitly distinguish randomly filled edges from observed edges during message passing, for instance, through a dedicated embedding or masking flag, so the model can learn to selectively trust structural context rather than treating all edges uniformly.

Extending SparserDiff to Larger Scales via Targeted Edge Sampling. While SparserDiff successfully eliminates the quadratic memory bottleneck associated with materializing non-existent edges during the forward noising process, our profiling reveals that the end-to-end training VRAM still scales quadratically. The dominant memory cost shifts from the sampler to the edge-level transformer activations and their backward-pass storage. This occurs because the model continues to merge a constant fraction of all possible node pairs as query edges into the computational graph, requiring $O(\lambda n^2)$ memory. Furthermore, simply reducing the sparsity parameter λ (e.g., below 0.1) is not a viable workaround, as empirical tests show this slows down the sampling process exponentially, rendering it computationally intractable. Therefore, the primary challenge for adapting this framework to larger scales is determining how to evaluate significantly fewer edges during the training step. A highly promising direction for future research is the transition from sampling “random edges” to adding “targeted edges.” By intelligently selecting a much smaller, structurally relevant subset of non-existent edges to evaluate, future models could bypass the $O(\lambda n^2)$ query-edge bottleneck entirely while preserving the high generation quality demonstrated by SparserDiff.

RedditWalk as a Large-Scale Benchmark. The RedditWalk dataset introduces a challenging and realistic evaluation setting that existing benchmarks do not provide. An important open question is whether generative models can learn not only the structural distribution of the sampled subgraphs, but

also semantic node types. Since the dataset exhibits strong homophily, a potential evaluation criterion is whether generated graphs preserve this alignment between node features and graph topology. Future work could formalize this as a benchmark task and use it to stress-test next-generation sparse graph diffusion models at graphs of 800+ node scales where current methods have not yet been thoroughly evaluated.

8 Conclusion

This project investigated the challenge of scaling generative diffusion models to large, sparse graphs, a problem that has remained largely unsolved due to the quadratic memory and computational costs inherent in existing approaches.

We introduced two complementary methods. SCALE-MGD applies discrete masked diffusion directly to a sparse edge-list representation, paired with a GNN-based architecture that avoids materializing the full adjacency matrix. This design achieves linear memory complexity $O(N + E)$, making it significantly more memory-efficient than existing approaches and enabling training at scales where all previous methods fail. SparserDiff, our second contribution, is a memory-efficient variant of SparseDiff that reformulates the forward-noising process to avoid materializing the full set of non-existent edges. While the underlying edge-level transformer still imposes an $O(\lambda N^2)$ bottleneck at extreme scales, SparserDiff dramatically reduces the peak memory of the sampling phase and preserves the high generation quality of the original model.

Our experiments show that SCALE-MGD’s linear scaling is a genuine quantitative advantage over existing methods, but this currently comes at the cost of structural fidelity. Generated graphs show significant misalignment with dataset distributions across all benchmarks, and no chemically valid molecules were produced on Zinc250K. The flexibility of the sparse masked representation, while key to its scalability, may be the reason it is harder for the model to learn the underlying graph structure.

Finally, we constructed RedditWalk, a 65,000-node giant weakly connected component derived from the Reddit Hyperlink Network, enriched with 20 semantic node types. Unlike synthetic benchmarks, which carry no node features, and molecular datasets which inherently impose validity constraints, RedditWalk supports the generation of variable-sized graph datasets that preserve the underlying network’s properties. We hope RedditWalk proves a useful resource for evaluating future large-scale graph generation frameworks.

Together, these contributions take concrete steps toward a long-standing open problem: generative modeling of large, sparse graphs. SCALE-MGD demonstrates that linear memory scaling is achievable in a diffusion framework, SparserDiff demonstrates that scalability and generation quality need not be in

opposition, and RedditWalk provides a foundation for evaluating future models at scales and complexity levels that existing benchmarks do not support. Closing the remaining gap between SCALE-MGD’s scalability and the generation quality already demonstrated by SparserDiff is a clear and tractable direction for future work.

References

- Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. Digress: Discrete denoising diffusion for graph generation. *arXiv preprint arXiv:2209.14734*, 2022.
- Chenhao Niu, Yang Song, Jiaming Song, Shengjia Zhao, Aditya Grover, and Stefano Ermon. Permutation invariant graph generation via score-based generative modeling. *arXiv preprint arXiv:2003.00638*, 2020. URL <https://arxiv.org/abs/2003.00638>.
- Wei Huang, Meiyu Liang, Peining Li, Xu Hou, Yawen Li, Junping Du, Zhe Xue, and Zeli Guan. Diffusioncom: Structure-aware multimodal diffusion model for multimodal knowledge graph completion. *arXiv preprint arXiv:2504.06543*, 2025. URL <https://arxiv.org/abs/2504.06543>.
- Trung-Kien Nguyen and Yuan Fang. Diffusion-based negative sampling on graphs for link prediction. In *Proceedings of the ACM Web Conference 2024 (WWW ’24)*, 2024. doi: 10.1145/3589334.3645650. URL <https://arxiv.org/abs/2403.17259>.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.
- Liang Wang, Chao Song, Zhiyuan Liu, Yu Rong, Qiang Liu, Shu Wu, and Liang Wang. Diffusion models for molecules: A survey of methods and tasks. *arXiv preprint arXiv:2502.09511*, 2025. URL <https://arxiv.org/abs/2502.09511>. Submitted 13 Feb 2025.
- Han Huang, Leilei Sun, Bowen Du, and Weifeng Lv. Conditional diffusion based on discrete graph structures for molecular graph generation. *arXiv preprint arXiv:2301.00427*, 2023. URL <https://arxiv.org/abs/2301.00427>. Accepted by AAAI 2023.
- Xiaochen Zhang, Shuangxi Wang, Ying Fang, and Qiankun Zhang. Mg-diff: A novel molecular graph diffusion model for molecular generation and optimization. *PLOS ONE*, 2025. doi: 10.1371/journal.pone.0331450. URL <https://doi.org/10.1371/journal.pone.0331450>.
- Yiming Qin, Clément Vignac, and Pascal Frossard. Sparsediff: Sparse discrete diffusion for scalable graph generation. *Trans. Mach. Learn. Res.*, 2025, 2025. URL <https://openreview.net/forum?id=kuJ3lpxnVC>.
- Simon J.D. Prince. *Understanding Deep Learning*. The MIT Press, 2023. URL <http://udlbook.com>.
- Camille Bilodeau, Wengong Jin, Tommi Jaakkola, Regina Barzilay, and Klavs F Jensen. Generative models for molecular discovery: Recent advances and challenges. *Wiley Interdisciplinary Reviews: Computational Molecular Science*, 12(5):e1608, 2022.
- Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. Diffuseq: Sequence to sequence text generation with diffusion models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=jQj-rLVXsj>.
- Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272. PMLR, 2017. URL <https://arxiv.org/abs/1704.01212>.

- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations*, 2021. URL <https://arxiv.org/abs/2006.05205>.
- Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020.
- Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform badly for graph representation? In *Advances in Neural Information Processing Systems*, volume 34, pages 28877–28888, 2021.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. URL <https://arxiv.org/abs/1312.6114>.
- Martin Simonovsky and Nikos Komodakis. Graphvae: Towards generation of small graphs using variational autoencoders. In *Artificial Neural Networks and Machine Learning-ICANN 2018*, pages 412–422. Springer, 2018. URL <https://arxiv.org/abs/1709.04108>.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, volume 27, 2014. URL <https://arxiv.org/abs/1406.2661>.
- Nicola De Cao and Thomas Kipf. Molgan: An implicit generative model for small molecular graphs. *arXiv preprint arXiv:1805.11973*, 2018. URL <https://arxiv.org/abs/1805.11973>.
- Aleksandar Bojchevski, Oleksandr Shchur, Daniel Zügner, and Stephan Günnemann. Netgan: Generating graphs via random walks. In *International Conference on Machine Learning*, pages 610–619. PMLR, 2018. URL <https://arxiv.org/abs/1803.00816>.
- Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. In *Advances in neural information processing systems*, volume 29, 2016.
- Lantao Yu, Weinan Zhang, Jun Wang, and Yong Yu. Seqgan: Sequence generative adversarial nets with policy gradient. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.
- Jiaxuan You, Rex Ying, Xiang Ren, William Hamilton, and Jure Leskovec. Graphrnn: Generating realistic graphs with deep auto-regressive models. In *International Conference on Machine Learning*, pages 5708–5717. PMLR, 2018. URL <https://arxiv.org/abs/1802.08773>.
- Yair Schiff, Subham Sekhar Sahoo, Hao Phung, Guanghan Wang, Sam Boshar, Hugo Dalla-Torre, Bernardo P. de Almeida, Thomas Pierrot, Alexander Rush, and Volodymyr Kuleshov. Simple guidance mechanisms for discrete diffusion models. *arXiv preprint, arXiv:2412.10193*, 2025. URL <https://arxiv.org/abs/2412.10193>.
- Lingkai Kong, Jiaming Cui, Haotian Sun, Yuchen Zhuang, B. Aditya Prakash, and Chao Zhang. Autoregressive diffusion model for graph generation. *arXiv preprint, arXiv:2307.08849*, 2023. URL <https://arxiv.org/abs/2307.08849>.
- Shu Yin, Dongpeng Hou, Lianwei Wu, Xianghua Li, and Chao Gao. A prior-based discrete diffusion model for social graph generation. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25)*, pages 3570–3578, 2025. doi: 10.24963/ijcai.2025/397. URL <https://doi.org/10.24963/ijcai.2025/397>.
- Jaehyeong Jo, Seul Lee, and Sung Ju Hwang. Score-based generative modeling of graphs via the system of stochastic differential equations. In *Proceedings of the 39th International Conference on Machine Learning (ICML-22)*, 2022. URL <https://arxiv.org/abs/2202.02514>. arXiv preprint arXiv:2202.02514.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. *arXiv preprint arXiv:2406.04329*, 2024. URL <https://arxiv.org/abs/2406.04329>.

- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Karolis Martinkus, Andreas Loukas, Nathanaël Perraudin, and Roger Wattenhofer. Spectre: Spectral conditioning helps to overcome the expressivity limits of one-shot graph generators. *arXiv preprint arXiv:2204.01613*, 2022. URL <https://arxiv.org/abs/2204.01613>.
- Paul D Dobson and Andrew J Doig. Distinguishing enzyme structures from non-enzymes without alignments. *Journal of Molecular Biology*, 2003. URL <https://pubmed.ncbi.nlm.nih.gov/12850146/>.
- Srijan Kumar, William L Hamilton, Jure Leskovec, and Dan Jurafsky. Community interaction and conflict on the web. In *Proceedings of the 2018 World Wide Web Conference*, pages 933–943, 2018. URL <https://arxiv.org/abs/1803.03697>.
- Jure Leskovec and Andrej Krevl. Snap datasets: Stanford large network dataset collection, 2014. URL <http://snap.stanford.edu/data>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. URL <https://arxiv.org/abs/1810.04805>.